

生成流网络作为熵正则化的 RL

Daniil Tiapkin*

CMAF - CNRS - École polytechnique
- Institut Polytechnique de Paris,
LMO, CNRS, Université Paris-Saclay,
HSE University

Nikita Morozov*

HSE University

Alexey Naumov

HSE University,
Steklov Mathematical Institute
Russian Academy of Sciences

Dmitry Vetrov

Constructor University, Bremen

摘要

最近提出的生成流网络 (GFlowNets) 是一种训练策略的方法, 该策略通过一系列动作按给定奖励的比例概率抽样组合离散对象。GFlowNets 利用了问题的顺序性质, 与强化学习 (RL) 有相似之处。我们的工作将 RL 和 GFlowNets 之间的联系扩展到一般情况。我们展示了如何将学习生成流网络的任务重新定义为具有特定奖励和正则化结构的熵正则化 RL 问题, 并且有效地进行。此外, 我们通过在几个概率建模任务中应用标准软 RL 算法来训练 GFlowNet, 证明了这种重新定义的实际效率。与之前报道的结果相反, 我们展示了熵 RL 方法可以与现有的 GFlowNet 训练方法竞争。这一视角为将 RL 原则直接整合到生成流网络领域开辟了一条道路。

根据未知归一化常数给出的概率质量函数, 在一个复杂的离散空间中进行操作。GFlowNets 的基本假设是该空间具有额外的顺序结构: 任何对象都可以通过从初始状态开始的一系列动作生成, 这在“不完整”对象之间诱导了一个有向无环图。本质上, GFlowNet 旨在学习一种随机策略, 选择动作并在该图中导航, 以便最终对象的分布与期望的分布相匹配。Bengio 等人 (2021) 的工作动机之一是分子生成: GFlowNet 从一个空分子开始, 并学习一种策略, 依次添加各种模块, 以匹配特定分子空间上的期望分布。

问题陈述与强化学习 (RL, Sutton 和 Barto, 2018) 有许多相似之处, 事实上, 现有的生成流网络算法经常借鉴 RL 的思想。特别是, 原始的 GFlowNet 训练方法——流匹配 (Bengio 等, 2021) 受到了时间差分学习 (Sutton, 1988) 的启发。接下来, RL 中的成熟技术成为开发新型 GFlowNet 技术的重要灵感来源: 子轨迹平衡损失 (Madan 等, 2023) 受到 TD(λ) 的启发; 优先经验回放的概念 (Lin, 1992; Schaul 等, 2016) 在 (Shen 等, 2023; Vemgal 等, 2023) 中被积极应用; RL 的分布视角 (Bellemare 等, 2023) 也出现在 GFlowNet 文献中 (Zhang 等, 2023)。此外, 各种 RL 探索技术 (Osband 等, 2016; Ostrovski 等, 2017; Burda 等, 2019) 最近被改编到 GFlowNet 框架中 (Pan 等, 2023; Rector-Brooks 等, 2023)。

1 引言

生成流网络 (GFlowNet, Bengio 等人, 2021) 是一种旨在从一个 * 等贡献。第 27 届国际人工智能与统计会议 (AISTATS) 2024, 西班牙瓦伦西亚。PMLR: 第 238 卷。
版权所有: 2024 作者。

这些技术的大量来源引发了一个关于 RL 和 GFlowNets 之间更严格联系的问题。不幸的是，仅以回报最大化为目标的经典 RL 无法充分解决采样问题，因为最终的最优策略是确定性的或接近确定性的。然而，存在一种熵正则化的 RL 范式 (Fox 等, 2016; Haarnoja 等, 2017; Schulman 等, 2017a) (也称为 MaxEnt RL 和软 RL)，其最终目标是找到一个不仅最大化回报而且接近均匀分布的策略，从而导致最优策略的能量表示 (Haarnoja 等, 2017)。

Bengio 等 (2021) 的作者直接建立了 MaxEnt RL 和 GFlowNets 之间的联系，在自回归生成设置中，或者等价地，在相应的生成图是树的情况下。然而，所提出的简化不适用于一般图的情况，并导致从高度偏置的分布生成。随后，Bengio 等 (2023) 的作者得出结论，这种直接联系仅在树结构设置下成立。

在我们的工作中，我们反驳了这一观点，并证明了可以将 GFlowNet 的学习问题简化为熵正则化的 RL，而不仅仅是树结构图的设置。

我们的主要推论是在 GFlowNet 框架内可以直接应用现有的强化学习技术，而无需额外适应。

我们强调我们的主要贡献如下：

- 我们通过直接约简建立了 GFlowNet 与熵正则化强化学习之间的联系；
- 我们将详细平衡 (Bengio 等, 2023 年) 和轨迹平衡 (Malkin 等, 2022 年) 算法表示为现有的软强化学习算法；
- 我们展示了如何通过传输 SoftDQN (Haarnoja 等, 2017 年) 和 MünchhausenDQN (Vieillard 等, 2020b 年) 到 GFlowNet 领域来实际应用所提出的约简；
- 最后，我们提供了对理论结果的实验验证，并表明 MaxEnt 强化学习算法可以具有竞争力甚至超越已建立的 GFlowNet 训练方法，这与一些先前报告的结果 (Bengio 等, 2021 年) 相反。

源代码: github.com/d-tiapkinn/gflownet-rl。

2 背景知识

2.1 生成流网络

在本节中，我们将提供一些关于 GFlowNet 的基本定义和背景知识。以下陈述的所有证明可以在 Bengio 等 (2021 年) 和 Bengio 等 (2023 年) 中找到。

遵循的声明的证明可以在 Bengio 等 (2021 年) 和 Bengio 等 (2023 年) 中找到。

假设我们有一个有限离散空间 $\mathcal{X}$ ，并给定一个对于每个 $x \in \mathcal{X}$ 的黑盒非负奖励函数 $\mathcal{R}(x)$ 。我们的目标是构建和训练一个模型，该模型将从概率分布 $\mathcal{R}(x)/Z$ 中按奖励比例采样对象 x ，其中 Z 是一个未知的归一化常数。

有向无环图上的流 GFlowNets 的生成过程被视为一系列构造性动作，我们从一个“空”对象开始，并在每一步添加一些新组件。为了正式描述这一过程，引入了一个有限的有向无环图 (DAG) $\mathcal{G} = (\mathcal{S}, \mathcal{E})$ ，其中 $\mathcal{S}$ 是状态空间， $\mathcal{E} \subseteq \mathcal{S} \times \mathcal{S}$ 是一组边。对于每个状态 $s \in \mathcal{S}$ ，我们定义一组可能的下一步状态的动作集 $\mathcal{A}_s$ ： $s' \in \mathcal{A}_s \Leftrightarrow s \rightarrow s' \in \mathcal{E}$ 。每个动作对应于向对象添加一些新组件 s ，将其转换为一个新的对象 s' 。如果 $s' \in \mathcal{A}_s$ ，我们说 s' 是 s 的子节点，而 s 是 s' 的父节点。有一个特殊的初始状态 $s_0 \in \mathcal{S}$ 与一个“空”对象相关联，它是唯一没有入边的状态。所有其他状态都可以从 s_0 到达，终端状态集 (没有出边的状态) 直接对应于感兴趣的初始空间 $\mathcal{X}$ 。

记 $\mathcal{T}$ 为图 $\tau = (s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_{n_\tau})$ 中所有完整轨迹的集合，其中 n_τ 是轨迹长度， $s_i \rightarrow s_{i+1} \in \mathcal{E}$ ， s_0 始终是初始状态，而 s_{n_τ} 始终是终端状态，因此 $s_{n_\tau} \in \mathcal{X}$ 。结果，任何完整的轨迹都可以视为从 s_0 开始构建对应于 s_{n_τ} 的对象的一系列动作。轨迹流是任意非负函数 $\mathcal{F}: \mathcal{T} \rightarrow \mathbb{R}_{\geq 0}$ 。如果 $\mathcal{F}$ 不恒等于零，它可以用来引入关于完整轨迹的概率分布：

$$\mathcal{P}(\tau) \triangleq \frac{1}{Z_{\mathcal{F}}} \mathcal{F}(\tau), \quad Z_{\mathcal{F}} \triangleq \sum_{\tau \in \mathcal{T}} \mathcal{F}(\tau).$$

然后，引入状态和边的流作为

$$\mathcal{F}(s) \triangleq \sum_{\tau \ni s} \mathcal{F}(\tau), \quad \mathcal{F}(s \rightarrow s') \triangleq \sum_{\tau \ni (s \rightarrow s')} \mathcal{F}(\tau).$$

那么， $\mathcal{F}(s)/Z_{\mathcal{F}}$ 对应于随机轨迹包含状态 s 的概率，而 $\mathcal{F}(s \rightarrow s')/Z_{\mathcal{F}}$ 对应于随机轨迹包含边 $s \rightarrow s'$ 的概率。这些定义还意味着 $Z_{\mathcal{F}} = \mathcal{F}(s_0) = \sum_{x \in \mathcal{X}} \mathcal{F}(x)$ 。

记 $\mathcal{P}(x)$ 为 x 是轨迹从 $\mathcal{P}(\tau)$ 中采样得到的终端状态的概率。如果所有终端状态都满足奖励匹配约束

$x \in \mathcal{X}$:

$$\mathcal{F}(x) = \mathcal{R}(x), \quad (1)$$

$\mathcal{P}(x)$ 将等于 $\mathcal{R}(x)/Z$ 。因此, 通过采样轨迹, 我们可以从感兴趣的分布中采样对象。

马尔可夫流 为了实现高效地采样轨迹, 实际中考虑的轨迹流族被限定为马尔可夫流。一个轨迹流 $\mathcal{F}$ 被称为马尔可夫流如果 $\forall s \in \mathcal{S} \setminus \mathcal{X}$ 存在分布 $\mathcal{P}_F(-|s)$ 覆盖 s 的子节点, 使得 $\mathcal{P}$ 可以分解为

$$\mathcal{P}(\tau = (s_0 \rightarrow \dots \rightarrow s_{n_\tau})) = \prod_{t=1}^{n_\tau} \mathcal{P}_F(s_t | s_{t-1}).$$

这意味着可以通过迭代地从 $\mathcal{P}_F(-|s_i)$ 采样下一个状态来采样轨迹 (顺序构造一个对象)。等价地, $\mathcal{F}$ 是马尔可夫流如果 $\forall s \in \mathcal{S} \setminus \{s_0\}$ 存在分布 $\mathcal{P}_B(-|s)$ 覆盖 s 的父节点, 使得 $\forall x \in \mathcal{X}$

$$\mathcal{P}(\tau = (s_0 \rightarrow \dots \rightarrow s_{n_\tau}) | s_{n_\tau} = x) = \prod_{t=1}^{n_\tau} \mathcal{P}_B(s_{t-1} | s_t),$$

这意味着结束于 x 的轨迹可以通过迭代地从 $\mathcal{P}_B(-|s_i)$ 采样前一个状态来采样。如果 $\mathcal{F}$ 是马尔可夫流, 则 $\mathcal{P}_F$ 和 $\mathcal{P}_B$ 可以唯一定义为

$$\mathcal{P}_F(s'|s) = \frac{\mathcal{F}(s \rightarrow s')}{\mathcal{F}(s)}, \quad \mathcal{P}_B(s|s') = \frac{\mathcal{F}(s \rightarrow s')}{\mathcal{F}(s')}. \quad (2)$$

我们将 $\mathcal{P}_F$ 和 $\mathcal{P}_B$ 分别称为前向策略和后向策略。细致平衡约束是它们之间的关系:

$$\mathcal{F}(s)\mathcal{P}_F(s'|s) = \mathcal{F}(s')\mathcal{P}_B(s|s'). \quad (3)$$

轨迹平衡约束说明对于任何完整的轨迹

$$\prod_{t=1}^{n_\tau} \mathcal{P}_F(s_t | s_{t-1}) = \frac{\mathcal{F}(s_{n_\tau})}{Z_{\mathcal{F}}} \prod_{t=1}^{n_\tau} \mathcal{P}_B(s_{t-1} | s_t). \quad (4)$$

因此任何马尔可夫流都可以唯一确定

$Z_{\mathcal{F}}$ 及 $\mathcal{P}_F(-|s) \forall s \in \mathcal{S} \setminus \mathcal{X}$ 或者从 $\mathcal{F}(x) \forall x \in \mathcal{X}$ 及 $\mathcal{P}_B(-|s) \forall s \in \mathcal{S} \setminus \{s_0\}$.

训练 GFlowNets 本质上, GFlowNet 是一个模型, 它参数化固定 DAG 上的马尔可夫流, 并通过最小化某个目标进行训练。目标的选择应使得, 如果模型能够参数化任何马尔可夫流, 则全局最小化该目标将导致生成满足 (1) 的流。因此, 训练后的 GFlowNet 的前向策略可以用来作为奖励分布的采样器。两个广泛使用的训练目标是: 轨迹平衡 (Trajectory Balance, TB)。由 Malkin 等人 (2022) 引入, 基于轨迹平衡约束 (4)。

具有参数 θ 的模型预测归一化常数 Z_θ 、前向策略 $\mathcal{P}_F(-|s, \theta)$ 和后向策略 $\mathcal{P}_B(-|s, \theta)$ 。轨迹平衡目标针对每个完整的轨迹 $\tau = (s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_{n_\tau} = x)$ 定义如下:

$$\mathcal{L}_{TB}(\tau) = \left(\log \frac{Z_\theta \prod_{t=1}^{n_\tau} \mathcal{P}_F(s_t | s_{t-1}, \theta)}{\mathcal{R}(x) \prod_{t=1}^{n_\tau} \mathcal{P}_B(s_{t-1} | s_t, \theta)} \right)^2. \quad (5)$$

细致平衡 (Detailed Balance, DB)。由 Bengio 等人 (2023) 引入, 基于细致平衡约束 (3)。具有参数 θ 的模型预测状态流 $\mathcal{F}_\theta(s)$ 、前向策略 $\mathcal{P}_F(-|s, \theta)$ 和后向策略 $\mathcal{P}_B(-|s, \theta)$ 。需要注意的是, 这种参数化是冗余的, 在训练过程中这些组件不一定彼此兼容。细致平衡目标针对每条边 $s \rightarrow s' \in \mathcal{E}$ 定义如下:

$$\mathcal{L}_{DB}(s, s') = \left(\log \frac{\mathcal{F}_\theta(s)\mathcal{P}_F(s'|s, \theta)}{\mathcal{F}_\theta(s')\mathcal{P}_B(s|s', \theta)} \right)^2. \quad (6)$$

对于终端状态 $\mathcal{F}_\theta(x)$ 在目标中替换为 $\mathcal{R}(x)$ 。

在训练过程中, 所选的目标是在从训练策略采样的完整轨迹上进行随机优化的 (该训练策略通常取为 $\mathcal{P}_F$ 、其温度版本或 $\mathcal{P}_F$ 与均匀分布的混合)。值得注意的是, 对于所有目标, $\mathcal{P}_B(-|s)$ 可以被训练或固定, 例如在整个父节点上均匀分布的 s 。当反向策略固定时, 相应的正向策略由 $\mathcal{P}_B$ 和 $\mathcal{R}$ 唯一确定。

在其他现有目标中, 最初提出的流匹配目标 (Bengio 等人, 2021 年) 被证明比时间平衡 (TB) 和差异平衡 (DB) (Malkin 等人, 2022; Madan 等人, 2023) 表现更差。最近, 子轨迹平衡 (SubTB) (Madan 等人, 2023) 作为 TB 和 DB 的泛化被引入, 并显示在适当调整时具有更好的性能。

2.2 软强化学习

本节将描述常规强化学习的主要方面及其熵正则化版本, 也称为最大熵 (MaxEnt) 和软强化学习 (Soft RL)。

马尔可夫决策过程 RL 的主要理论对象是马尔可夫决策过程 (MDP), 定义为一个元组 $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma, s_0)$, 其中 $\mathcal{S}$ 和 $\mathcal{A}$ 是有限的状态空间和动作空间, $\mathcal{P}$ 是一个马尔可夫转移核, r 是一个有界确定性奖励函数, $\gamma \in [0, 1]$ 是一个折扣因子, 而 s_0 是一个固定的初始状态。此外, 对于每个状态 s , 我们定义一组可能的动作作为 $\mathcal{A}_s \subseteq \mathcal{A}$ 。RL 代理通过策略 π 与 MDP 交互, 该策略将每个状态 s 映射到可能动作的概率分布。

熵正则化 RL 经典 RL 中代理的性能测度是一个值，定义为遵循策略 π 从状态 s 开始生成的轨迹上的预期（折扣）奖励之和。相比之下，在熵正则化的强化学习（Neu 等人，2017；Geist 等人，2019）中，值被推广了香农熵：

$$V_{\lambda}^{\pi}(s) \triangleq \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \lambda \mathcal{H}(\pi(s_t))) | s_0 = s \right], \quad (7)$$

其中 λ 是正则化系数， $\mathcal{H}$ 是香农熵函数， $a_t \sim \pi(s_t), s_{t+1} \sim P(s_t, a_t)$ 对所有 $t \geq 0$ 成立，并且期望是关于这些随机变量计算的。类似地，我们可以定义正则化的 Q 值 Q^{π} 为在给定初始 (s, a) 状态 $s_0 = s$ 和动作的情况下，奖励的期望（折扣）和香农熵之和。正则化的最优策略 π_{λ}^* 是一个最大化 $V_{\lambda}^{\pi}(s)$ 对于任 $a_0 = a$ 何初始状态 s 的策略。

Soft Bellman 方程 设 V_{λ}^* 和 Q_{λ}^* 分别为最优策略 π_{λ}^* 的值和 Q 值。然后，Haarnoja 等人（2017）的定理 1 和 2 暗示了以下系统关系对于任意 $s \in \mathcal{S}, a \in \mathcal{A}_s$

$$\begin{aligned} Q_{\lambda}^*(s, a) &\triangleq r(s, a) + \gamma \mathbb{E}_{s' \sim P(s, a)} [V^*(s')] \\ V_{\lambda}^*(s) &\triangleq \text{LogSumExp}_{\lambda}(Q_{\lambda}^*(s, \cdot)), \end{aligned} \quad (8)$$

其中 $\text{LogSumExp}_{\lambda}(x) \triangleq \lambda \log(\sum_{i=1}^d e^{x_i/\lambda})$ 为相对于最优 Q 值 λ -Softmax 函数的策略 $\pi_{\lambda}^*(s) = \text{Softmax}_{\lambda}(Q_{\lambda}^*(s, \cdot))$ ，其中

$$[\text{Softmax}_{\lambda}(x)]_i \triangleq e^{x_i/\lambda} / (\sum_{j=1}^d e^{x_j/\lambda}), \text{ 或者,}$$

可选地

$$\pi_{\lambda}^*(a|s) = \exp\left(\frac{1}{\lambda}(Q_{\lambda}^*(s, a) - V_{\lambda}^*(s))\right). \quad (9)$$

注意到当 $\lambda \rightarrow 0$ 我们得到著名的贝尔曼方程（参见 Sutton 和 Barto(2018)），因为 $\text{LogSumExp}_{\lambda}$ 近似于最大值，并且 Softmax_{λ} 近似于 $\arg \max$ 上的均匀分布。

软深度 Q 网络 解决离散动作空间中强化学习问题的经典方法之一是著名的深度 Q 网络算法（DQN）（Mnih 等，2015）。下文我们将介绍该算法在熵正则化强化学习问题中的泛化形式，称为 SoftDQN（在连续控制环境中也被称为 SoftQ 学习（Fox 等，2016；Haarnoja 等，2017；Schulman 等，2017a））。

SoftDQN 算法的目标是通过一个由神经网络参数化的 Q 值来逼近软贝尔曼方程（8）的解。设 Q_{θ} 为在线 Q 网络，表示当前对最优正则化 Q 值的近似。另设 $\pi_{\theta} = \text{Softmax}_{\lambda}(Q_{\theta})$ 为一种策略，该策略近似最优策略。然后，类似于 DQN，智能体使用策略 π_{θ} 与马尔可夫决策过程进行交互（附加

ε -贪婪探索甚至不使用的情况下进行收集

转换 (s_t, a_t, r_t, s_{t+1}) 对 $r_t = r(s_t, a_t)$

在一个回放缓冲区中 $\mathcal{B}$ 。然后 SoftDQN 对回归损失执行随机梯度下降

$$\mathcal{L}_{\text{SoftDQN}}(\mathcal{B}) = \text{目标 Q-value} \mathbb{E}_{\mathcal{B}}[(Q_{\theta}(s_t, a_t) - y_{\text{SoftDQN}}(r_t, s_{t+1}))^2]$$

$$y_{\text{SoftDQN}}(r_t, s_{t+1}) = r_t + \gamma \text{LogSumExp}_{\lambda}(Q_{\bar{\theta}}(s_{t+1}, \cdot)), \quad (10)$$

其中 $Q_{\bar{\theta}}$ 是一个具有参数 $\bar{\theta}$ 的目标 Q 网络，这些参数周期性地从在线 Q 网络的参数 θ 复制而来。此外，还存在其他算法，它们将传统的强化学习推广到了软强化学习，例如著名的 SAC（Haarnoja 等人，2018）及其离散版本（Christodoulou，2019）。

值得一提的是另一种算法，如 Munchausen DQN（M-DQN，Vieillard 等人，2020b），它利用类似于 SoftDQN 的正则化结构，但通过当前策略增强目标：

$$y_{\text{M-DQN}}(s_t, a_t, r_t, s_{t+1}) = y_{\text{SoftDQN}}(r_t, s_{t+1}) + \alpha \lambda \log \pi_{\bar{\theta}}(a_t | s_t).$$

结果表明，该方案等价于（在重新参数化下）近似镜像下降价值迭代（Vieillard 等人，2020a）方法，用于求解 $(1 - \alpha)\lambda$ 熵正则化 MDP，并附加关于目标策略的 KL 正则化，系数为 $\alpha\lambda$ （Vieillard 等人，2020b，定理 1）。

3 REPRESENTATION OF GFLOWNETS AS SOFT RL

本节描述了 GFlowNets 与熵正则化强化学习之间的深层联系。

3.1 先前工作

GFlowNet 解决的问题类型是顺序的，这自然形成了并行处理，而流匹配损失的主要启发来源于时间差学习（Bengio 等人，2021）。

除了启发之外，在开创性的论文（Bengio 等人，2021）中建立了 GFlowNets 与 RL 之间的正式联系。具体而言，作者展示了可以通过在树状有向无环图设置中应用最大熵强化学习来学习 GFlowNet 的前向策略。然而，在一般情况下，作者声称最大熵强化学习是以概率与 $n(x)\mathcal{R}(x)$ 成比例的方式进行采样的，其中 $n(x)$ 是从终端状态的路径数量 $x \in \mathcal{X}$ ，因此这些方法应具有高度偏差。因此，（Bengio 等人，2023）的作者声称，最大熵强化学习与 GFlowNet 之间的等价关系仅在底层有向无环图具有树结构时成立。

此外, 可以将 GFlowNet 学习问题视为一个凸 MDP 问题 (Zahavy 等, 2021), 目标是直接最小化 $KL(d^\pi \| \mathcal{R}/Z)$, 其中 d^π 是由策略 π 在终止状态上诱导出的分布。然而, 正如 Bengio 等人 (2023) 所声称的, 基于强化学习的算法需要高质量地估计分布 d^π , 而 Zahavy 等人 (2021) 的方法需要在学习过程中对变化的策略进行这种估计。类似的问题出现在深度强化学习中基于计数的探索泛化 (Bellemare 等, 2016; Ostrovski 等, 2017; Saade 等, 2023)。

3.2 通过 MDP 构造直接简化

本节展示如何将给定奖励函数 $\mathcal{R}$ 和反向策略 $\mathcal{P}_B$ 下学习 GFlowNet 正向策略 $\mathcal{P}_F$ 的问题重新表述为软强化学习问题。

首先, 对于任何有向无环图 $\mathcal{G} = (\mathcal{S}, \mathcal{E})$ 及其终端状态集合 $\mathcal{X}$, 我们可以定义相应的确定性控制马尔可夫过程 $(\mathcal{S}', \mathcal{A}, P)$, 并添加一个吸收状态 s_f 。形式上, 构造如下。首先, 选择状态空间 $\mathcal{S}'$ 作为相应无向无环图 $\mathcal{G}$ 的状态空间, 并添加一个吸收状态 s_f : $\mathcal{S}' = \mathcal{S} \cup \{s_f\}$ 。接下来, 不定义完整的动作空间 $\mathcal{A}$, 而是对于任意 s , 选择 $\mathcal{A}_s \subseteq \mathcal{S}$ 作为状态 s 在 $\mathcal{S} \setminus \mathcal{X}$ 中的动作集, 并定义 $\mathcal{A}_x \triangleq \{s_f\}$ 对于任意 $x \in \mathcal{X} \cup \{s_f\}$ 。换句话说, 每个终端状态只有一个动作指向 s_f , 而 s_f 本身有一个循环。然后, 我们可以定义 $\mathcal{A}$ 为所有 $\mathcal{A}_s$ 的并集, 对于任意 $s \in \mathcal{S}'$ 。最后, 确定性马尔可夫核的构造很简单:

$P(s'|s, a) = \mathbb{1}\{s' = a\}$ 因为 a 对应于给定相应动作后的下一个状态在 $\mathcal{S}$ 中。随后, 我们将写作 s' 代替 a , 以强调这一联系。

定理 1. 设 $\mathcal{G} = (\mathcal{S}, \mathcal{E})$ 是有向无环图, 其终端状态集合为 $\mathcal{X}$, 设 $\mathcal{P}_B$ 为固定反向策略, $\mathcal{R}$ 为 GFlowNet 奖励函数。

Let $\mathcal{M}_\mathcal{G} = (\mathcal{S}', \mathcal{A}, P, r, \gamma, s_0)$ be an MDP with $\mathcal{S}', \mathcal{A}, P$ 从 $\mathcal{G}$ 构造而来, 附加一个吸收状态 s_f , $\gamma = 1$, 并且 MDP 奖励定义如下

$$r(s, s') = \begin{cases} \log \mathcal{P}_B(s|s') & s \notin \mathcal{X} \cup \{s_f\}, \\ \log \mathcal{R}(s) & s \in \mathcal{X}, \\ 0 & s = s_f \end{cases} \quad (11)$$

那么, 正则化的最优策略 $(s'|s) \cdot \pi^*$ 系数为 $\lambda = 1$ 的 MDP 等于 $\mathcal{P}_F(s'|s)$ 对于所有情况 $s \in \mathcal{S} \setminus \mathcal{X}, s' \in \mathcal{A}_s$;

Moreover, $\forall s \neq s_f, s' \neq s_f$ regularized optimal value $V_1^*(s)$ and Q -value $Q_1^*(s, s')$ coincide with $\log \mathcal{F}(s)$ and $\log \mathcal{F}(s \rightarrow s')$ respectively, where $\mathcal{F}$ is the Markovian flow defined by $\mathcal{P}_B$ and the GFlowNet reward $\mathcal{R}$.

完整的证明在附录 B 给出, 并主要使用了软贝尔曼方程 (8) 和图的拓扑排序上的逆向归纳。

直觉. 给定一个固定的 GFlowNet 逆向策略, 最大熵强化学习中带有系数 $\lambda = 1$ 和调整后的奖励的最优策略与 GFlowNet 正向策略相同。关键洞见在于通过固定的逆向策略引入中间奖励, 以便对通向同一终端状态的不同路径的概率进行归一化。

注释 1. 在树结构 DAG 的情形下 $\mathcal{G}$, 我们注意到我们的结果表明只需定义终端状态的奖励 $\log \mathcal{R}$ 即可, 因为任何逆向策略 $\mathcal{P}_B$ 都满足 $\log \mathcal{P}_B(s|s') = 0$ 。因此, 我们重现了 Bengio 等人 (2021) 提出的命题 1(a,b) 的结果, 参见 (Bengio 等, 2023)。关于部分 (c) 的结果, 我们构造的 MDP 与 Bengio 等人 (2021) 构造的关键区别在于非终端状态的非平凡奖励“补偿”了出现在 (Bengio 等, 2021) 中的乘数 $n(x)$ 。

注释 2. 值得注意的是, 我们的推导解释了在 TB(见 (5))、DB(见 (6)) 和 SubTB(Madan 等, 2023) 损失中出现的对数缩放。之前, 这些对数的出现仅由数值原因解释 (Bengio 等, 2021, 2023)。

注记 3. 在定理 1 中设定奖励的选择下, 取 $\lambda \neq 1$ 将导致一个有偏差的策略。为了说明这一点, 注意到使用系数 $\lambda > 0$ 和奖励 r 解决软强化学习问题等价于使用系数 $\lambda' = 1$ 和新奖励解决软强化学习问题。

$$r' = r/\lambda \text{ since } V_\lambda^*(s_0; r) = \mathbb{E}[\sum_t r_t + \lambda \mathcal{H}(\pi(s_t))] = \lambda \mathbb{E}[\sum_t r_t / \lambda + \mathcal{H}(\pi(s_t))] = \lambda V_1^*(s_0; r/\lambda)$$

对于终端奖励, 这会导致重新缩放 GFlowNet 奖励 $R(x) \mapsto R^{1/\lambda}(x)$, 但所有中间奖励也会被重新缩放 $\log \mathcal{P}_B(s|s') \mapsto \log \mathcal{P}_B(s|s')/\lambda$, 从而产生偏差。然而, 在定理 1 中对 $\lambda \neq 1$ 的中间奖励乘以 λ 将导致一个从温控 GFlowNet 奖励分布中采样的策略。

注记 4. 我们强调与 Bengio 等人 (2021) 实验设置的两个主要差异: 他们应用了带有正则化系数 $\lambda < 1$ 的 PPO(Schulman 等人, 2017b) 和带有自适应正则化系数的 SAC(Haarnoja 等人, 2018)(而我们的构造需要 $\lambda = 1$), 并且将中间奖励设为零 (而我们将它们设为 $\log \mathcal{P}_B(s|s')$)。这导致收敛到一个有偏差的最终策略。

价值解释 值得注意的是, 在马尔可夫决策过程 MDP $\mathcal{M}_\mathcal{G}$ 中, 我们有一个明确的价值函数解释。但首先我们需要引入几个重要定义。对于确定性 MDP $\mathcal{M}_\mathcal{G}$ 中的策略 π , 其中所有轨迹长度为 $n_\tau \leq N$, 我们可以定义一个诱导轨迹分布如下

$$q^\pi(\tau) = \prod_{i=1}^N \pi(s_i | s_{i-1}) = \prod_{i=1}^{n_\tau} \pi(s_i | s_{i-1}),$$

其中 $\tau = (s_0 \rightarrow \dots \rightarrow s_N)$ ，给定 $s_N = s_f$ 和 $s_{n_\tau} \in \mathcal{X}$ 。最后一个恒等式成立是因为对于任何 $x \in \mathcal{X}$ ，都有 $\pi(s_f | s_f) = \pi(s_f | x) = 1$ 。此外，我们定义通过策略 π 采样的边缘分布为 $d^\pi(x) = \mathbb{P}_\pi[s_{n_\tau} = x]$ 。此外，对于固定的 GFlowNet 奖励 $\mathcal{R}$ 和反向策略 $\mathcal{P}_B$ ，我们定义一个对应的轨迹分布，稍作符号上的滥用

$$\mathcal{P}_B(\tau) = \frac{\mathcal{R}(s_{n_\tau})}{Z} \prod_{i=1}^{n_\tau} \mathcal{P}_B(s_{i-1} | s_i).$$

注意到通过轨迹平衡约束 (4) 和定理 1 $\mathcal{P}_B(\tau)$ 等于 $q^{\pi_1^*}(\tau)$ 。

命题 1. 设 $\mathcal{M}_G$ 是由定理 1 通过有向无环图 $\mathcal{G}$ 构造的马尔可夫决策过程，给定一个固定的逆向策略 $\mathcal{P}_B$ 和 GFlowNet 奖励函数 $\mathcal{R}$ 。对于任何策略 π ，其在初始状态 s_0 的正则化值 V_1^π 等于

$$V_1^\pi(s_0) = \log Z - \text{KL}(q^\pi \| \mathcal{P}_B).$$

证明思路是应用价值的直接定义 (7)。完整的证明见附录 B。

该命题的主要结论是，在规定的马尔可夫决策过程中最大化价值函数 $\mathcal{M}$ 等价于最小化由当前策略和反向策略诱导的轨迹分布之间的 KL 发散。此外，它表明所有接近最优的软 RL 策略引入的流具有相似的轨迹分布，并且具有相似的边缘分布：

$$V_1^*(s_0) - V_1^\pi(s_0) = \text{KL}(q^\pi \| \mathcal{P}_B) \geq \text{KL}(d^\pi \| \mathcal{R}/Z),$$

其中 $d^\pi(x)$ 是由策略诱导的终端状态的边缘分布，而 π 和 $\mathcal{R}(x)/Z$ 是我们希望从中采样的奖励分布，参见 (Cover, 1999, 定理 2.5.3)。换句话说，策略误差控制着分布近似误差，进一步证明了所提出的马尔可夫决策过程构造的合理性。

3.3 软 DQN 在 GFlowNet 领域的应用

在本节中，我们将解释如何将 SoftDQN 算法应用于 GFlowNet 训练。

正如第 2.2 节所述，在 SoftDQN 中，代理使用一个网络 Q_θ 来逼近最优的 Q 值，或者等效地，逼近边流的对数。为了与有向无环图交互，代理使用 Softmax 策略 $\pi_\theta(s) = \text{Softmax}_1(Q_\theta(s, \cdot))$ 。在交互过程中，代理将转换 (s_t, a_t, r_t, s_{t+1}) 收集到回放缓冲区 $\mathcal{B}$ 中，并使用带有奖励 $r_t = \log \mathcal{P}_B(s_t | s_{t+1})$ 的目标 (10) 优化回归损失 $\mathcal{L}_{\text{SoftDQN}}$ ，对于任意的 $s_t \notin \mathcal{X}$ 。

对于终端转移 $s_t \in \mathcal{X}$ 和 $s_{t+1} = s_f$ ，目标等于 $\log \mathcal{R}(s_t)$ 。具体而言，通过解释 $Q_\theta(s, s) = \log \mathcal{F}_\theta(s \rightarrow s')$ 我们可以

获得以下损失

$$\mathcal{L}_{\text{SoftDQN}}(\mathcal{B}) = \mathbb{E}_{\mathcal{B}} \left[\left(\log \frac{\mathcal{F}_\theta(s_t \rightarrow s_{t+1})}{\mathcal{P}_B(s_t | s_{t+1}) \mathcal{F}_{\bar{\theta}}(s_{t+1})} \right)^2 \right], \quad (12)$$

其中 $x) \forall x \in \mathcal{X}$ ，否则 $\mathcal{F}^-(s) = \sum_{s'} \mathcal{F}_{\bar{\theta}}(s \rightarrow s')$ 。

是目标网络的参数，该网络不时地更新权重 θ 。具体来说，这种损失可以从反向策略定义 (2) 推导出来，类似于从相应的恒等式 (3)-(4) 推导出 DB 和 TB 损失，其差异在于固定反向策略和使用目标网络 $\bar{\theta}$ 及回放缓冲区 $\mathcal{B}$ 。关于技术算法选择的讨论，请参见附录 C。

芒豪森 DQN 使用完全相同的表示，我们可以将 M-DQN (Vieillard 等人, 220 年 b) 应用于训练 GFlowNets。然而，与 SoftDQN 不同，它需要更仔细地选择 λ ，因为 M-DQN 解决了一个带有熵系数 $(1-\alpha)\lambda$ 的正则化问题。因此，需要选择一个非平凡的 $\lambda = 1/(1-\alpha)$ 来补偿芒豪森惩罚的影响。关于更详细的描述，请参见附录 C。

可学习的反向策略 上述方法适用于固定的反向策略，这是一个可行且广泛用于 GFlowNet 训练的选择 (Malkin 等人, 2022; Zhang 等人, 2022b; Deleu 等人, 2022; Malkin 等人, 2023; Madan 等人, 2023)，具有诸如更稳定的训练等优点。同时，拥有一个可训练的反向策略可以导致更快的收敛 (Malkin 等人, 2022)。我们的方法仍然缺乏对反向策略优化效果的理解，这是一个局限性。

然而，似乎可以将这种优化从博弈论的角度进行解释：一名玩家选择奖励（由逆向策略定义），以优化其目标，另一名玩家则试图猜测一个良好的正向策略来应对这些奖励。这一想法在凸 MDP 和最大熵探索中已有应用，参见 (Zahavy 等, 2021; Tiapkin 等, 2023)。本文将形式化推导留作未来研究的一个有前景的方向。

3.4 现有的通过软强化学习的 GFlowNets

在本节中，我们展示了 DB 和 TB GFlowNet 损失可以从现有的强化学习文献中推导出来，具体细节除外。

细致平衡作为对抗软 DQN 为了更直接地展示 SoftDQN 与 DB 算法之间的联系，本文首先介绍了一种在强化学习中广为人知的技术，即双网络架构 (Wang 等, 2016)。双网络架构不仅仅参数化 Q 值

架构提议采用一个具有两条流的神经网络来处理值 $V_\theta(s)$ 以及动作优势 $A_\theta(s, a)$ 。这两条流与原始的 Q 值相连作为 $Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a)$ 根据优势函数的定义。

该技术的核心思想在于，人们认为价值可以更快地被学习，并通过自举方法加速优势的学习（唐等，2023）。然而，正如开创性工作（王等，2016）所指出的，上述重参数化是不可识别的，因为优势不能唯一地由 Q 值定义。在经典强化学习设置中，建议减去优势的最大值。而在软强化学习设置中，减去优势的对数和指数 (log-sum-exp) 是有意义的，从而得到以下恒等式

$$Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a) - \log \sum_{a'} \exp(A_\theta(s, a')).$$

LogSumExp 这种表示的关键属性如下关系（在 GFlowNet 设置中，对于 $\lambda = 1$ ）：

$$\begin{aligned} \text{对数和指数 } \log \pi_\theta(a|s) &= A_\theta(s, a) - \log \sum_{a'} \exp(A_\theta(s, a')), \\ \log \pi_\theta(a|s) &= A_\theta(s, a) - \text{LogSumExp}_1(A_\theta(s, \cdot)). \end{aligned}$$

利用 GFlowNet 解释 $V_\theta(s) = \log \mathcal{F}(s)$ 和 $\pi_\theta(s'|s) = \mathcal{P}_F(s'|s, \theta)$ ，损失函数 (12) 转换为

$$\mathcal{L}_{\text{SoftDQN}}^{\text{Dueling}}(\mathcal{B}) = \mathbb{E}_{\mathcal{B}} \left[\left(\log \frac{\mathcal{P}_F(s_{t+1}|s_t, \theta) \mathcal{F}_\theta(s_t)}{\mathcal{P}_B(s_t|s_{t+1}) \mathcal{F}_\theta(s_{t+1})} \right)^2 \right],$$

这与 DB-损失 (6) 相吻合，只是缺少可学习的反向策略以及重放缓冲区和目标网络的使用。

轨迹平衡作为策略梯度 我们考虑策略梯度方法的设置：通过参数化策略空间 π_θ 由参数 θ 我们的目标是直接最大化 $V^{\pi_\theta}(s_0)$ 。策略梯度方法的核心是在 θ 上应用随机梯度上升以优化此目标。

然而，命题 1 和 (Malkin 等人，2023) 中的命题 1 的结合意味着

$$-\nabla_\theta V_1^{\pi_\theta}(s_0) = \nabla_\theta \text{KL}(q^{\pi_\theta} \| \mathcal{P}_B) = \frac{1}{2} \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \mathcal{L}_{\text{TB}}(\tau)],$$

其中 $\mathcal{L}_{\text{TB}}(\tau)$ 是 TB-损失 (5)。这意味着在策略上的 TB-损失的优化等同于使用 $\log Z_\theta$ 作为基线函数的策略梯度。此外，这种联系非常接近 Schulman 等人 (2017a) 所建立的软 Q 学习与策略梯度方法之间的联系。

4 实验

在本节中，我们评估了软 RL 方法在所提出的 GFlowNet 训练任务中的表现

范式并与先前的 GFlowNet 训练方法进行比较：轨迹平衡 (TB, Malkin 等，2022 年)，细致平衡 (DB, Bengio 等，2023 年) 和子轨迹平衡 (SubTB, Madan 等，2023 年)。在软强化学习方法中，为了简洁起见，本节仅展示了 M nchhausen DQN (M-DQN, Vieillard 等，2020b) 的结果，因为我们发现它相较于其他考虑的技术具有更好的性能。关于消融研究和额外的比较，请参阅附录 E。

4.1 超网格环境

我们从 Bengio 等人 (2021) 引入的合成超网格环境开始。这些环境足够小，使得归一化常数 Z 可以精确计算，并且训练出的策略可以高效地与奖励分布进行评估。

非终端状态集合是一个具有整数坐标的点集，位于一个维度为 D 的超立方体内，边长为

$H: \{(s^1, \dots, s^D) \mid s^i \in \{0, 1, \dots, H-1\}\}$ 。初始状态是 $(0, \dots, 0)$ ，对于每个非终端状态 s ，存在其副本 $s^\top$ 作为终端状态。对于非终端状态 s ，允许的操作包括将一个坐标值加 1 而不离开网格，以及终止操作 $s \rightarrow s^\top$ 。奖励在网格的角落附近有峰值，这些峰值之间由宽而低的凹槽隔开。我们研究了二维和四维网格，奖励参数取自 Malkin 等人 (2022) 的研究。为了衡量 GFlowNet 在训练过程中的性能，计算了真实奖励分布 $\mathcal{R}(x)/Z$ 与训练过程中最后 $2 \cdot 10^5$ 个样本的实证分布 $\pi(x)$ 之间的 L^1 距离（轨迹端点从 $\mathcal{P}_F$): $\frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} |\mathcal{R}(x)/Z - \pi(x)|$ 。需要注意的是，在 M-DQN 的情况下，我们有 $\mathcal{P}_F(\cdot|s) = \text{Softmax}_\lambda(Q_\theta(s, \cdot))$ 对于 $\lambda = 1/(1 - \alpha)$ 。所有模型都通过多层感知器参数化，该感知器接受一个热编码的 s 作为输入。更多细节请参见附录 D.1。

我们考虑两种基线情况：均匀分布和学习得到的 $\mathcal{P}_B$ 。由于我们的理论框架不支持学习得到的 $\mathcal{P}_B$ ，我们将 M-DQN 的所有情况下的基线固定为均匀分布。图 1 展示了结果。在均匀分布的情况下，M-DQN 比所有基线更快收敛 $\mathcal{P}_B$ 。训练过程 $\mathcal{P}_B$ 提高了基线的收敛速度，在某些情况下，SubTB 的表现优于 M-DQN。然而，即使对于学习得到的 $\mathcal{P}_B$, TB, DB 的收敛速度仍然慢于 M-DQN。

4.2 小分子生成

接下来，我们考虑 Bengio 等人 (2021) 提出的分子生成任务。目标是生成可结合 sEH（可溶性环氧化物水解酶）蛋白的配体。由图表示的分子通过

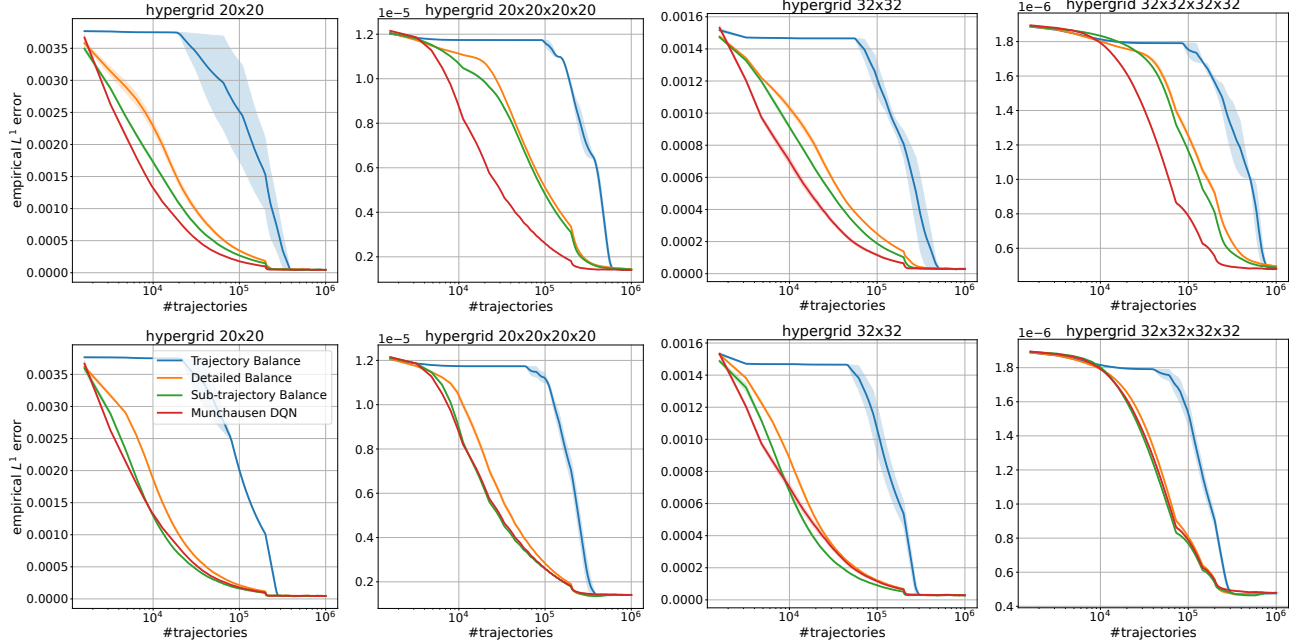


图 1: L^1 训练过程中目标与经验 GFlowNet 分布之间的距离。上行: $\mathcal{P}_B$ 对所有方法固定为均匀分布。下行: $\mathcal{P}_B$ 对基线进行学习, 并对 M-DQN 固定为均匀分布。均值和标准差基于 3 次运行计算得出。

顺序连接预定义构建块词汇表中的部分 (Jin 等人, 2020)。状态空间大小约为 10^{16} , 每个状态有 100 到 2000 个可能的动作。奖励基于对接预测 (Trott 和 Olson, 2010), 并以 Bengio 等人 (2021) 提供的预训练代理模型 $\hat{\mathcal{R}}$ 形式给出。此外, 引入了奖励指数超参数 β , 因此 GFlowNet 使用奖励 $\mathcal{R}(x) = \hat{\mathcal{R}}(x)^\beta$ 进行训练。

我们遵循 Madan 等人 (2023) 的设置。每个模型在不同的学习率下进行训练, 并且 β , $\mathcal{P}_B$ 对所有方法保持一致。为了衡量训练好的模型与目标分布的匹配程度, 在固定范围内的分子测试集中计算皮尔逊相关性, 该测试集的范围是从 $\log \mathcal{R}(x)$ 到 $\log \mathcal{P}_\theta(x)$, 其中后者是当 x 作为从 $\mathcal{P}_F$ 采样的轨迹结束时的对数概率, 其估计方式与 Madan 等人 (2023) 相同。此外, 我们跟踪每种方法在整个训练过程中捕获的 Tanimoto 分离模式的数量, 这与 Bengio 等人 (2021) 提出的方案一致。进一步的实验细节见附录 D.2。

结果展示在图 2 中。我们发现 M-DQN 在奖励相关性方面优于 TB 和 DB, 并且与 SubTB 表现相似 (或略差当 $\beta = 4$)。它还具有与 SubTB 类似的对学习率选择的鲁棒性。然而, M-DQN 在整个训练过程中发现的模式数量超过所有基线。

4.3 非自回归序列生成

最后, 我们考虑 Malkin 等人 (2022) 和 Madan 等人 (2023) 提出的比特序列生成任务。为了创建一个更难的 GFlowNet 学习任务, 具有非树状 DAG 结构, 我们修改了状态和动作空间, 允许类似于 Zhang 等人 (2022b) 所用方法的非自回归生成。

目标是生成固定长度的二进制字符串 $n = 120$ 。奖励由模式集指定 $M \subset \mathcal{X} = \{0, 1\}^n$ 并定义为 $\mathcal{R}(x) = \exp(-\min_{x' \in M} d(x, x'))$, 其中 d 表示汉明距离。我们按照 Malkin 等人 (2022) 的方法构建 M 并使用 $|M| = 60$ 。参数 k 作为轨迹长度和动作空间大小之间的权衡因素。对于变化的 $k | n$ 词汇表被定义为所有 k -位的词。生成从一个序列开始 n/k 特殊的“空”词汇 $\emptyset$, 在每一步中, 允许的操作是从词汇表中选择一个词来替换任何一个空位。因此, 状态空间由一系列序列组成 n/k 词语, 具有其中之一 $2^k + 1$ 每个位置可能的单词。终止状态是没有空单词的状态, 因此对应长度为 n 。

为了评估模型性能, 我们采用了 Malkin 等人 (2022) 和 Madan 等人 (2023) 用于此任务的两个度量: 1) 测试集上 $\mathcal{R}(x)$ 与 $\mathcal{P}_\theta(x)$ 之间的斯皮尔曼相关性 2) 训练过程中捕获的模式数量

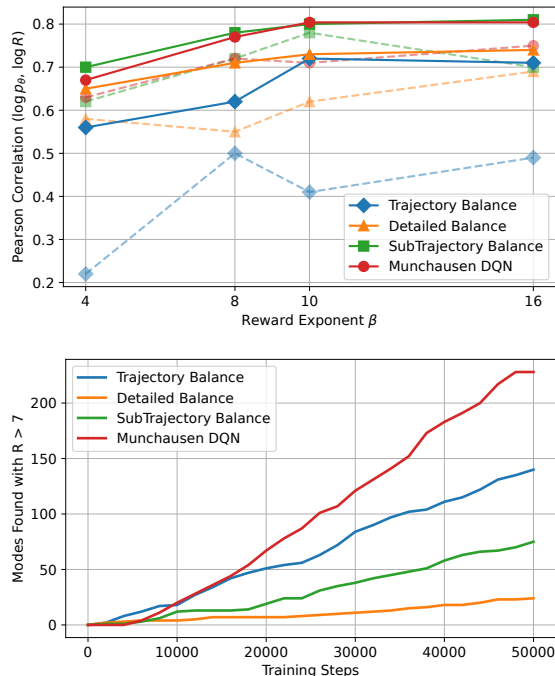


图 2：小分子生成结果。上图：每种方法在不同 $\beta \in \{4, 8, 10, 16\}$ 下测试集上的 $\log R$ 与 $\log P_\theta$ 之间的皮尔森相关性。实线表示在学习率选择下的最佳结果，虚线表示平均结果。下图：训练过程中发现的 Tanimoto 分离模式的数量，对于 $\beta = 10$ 为 $\hat{R} > 7.0$ 。

训练（从 M 生成的距离为 $\delta = 30$ 的候选比特串数量）。与 Malkin 等人 (2022) 的设置对比，在我们的案例中，由于通往每个 x 的路径数量众多，从 GFlowNet 中采样 x 的确切概率难以计算，因此我们采用了 Zhang 等人 (2022b) 提出的蒙特卡洛估计方法。在实验中，我们将 P_B 设定为均匀分布。更多细节请参见附录 D.3。

图 3 展示了结果。M-DQN 在奖励相关性方面与所有基线相似或更好，除了在 $k = 6$ 的情况下，它优于 TB 和 DB 但落后于 SubTB。在发现模式方面，它的表现优于 DB，与 TB 和 SubTB 相当。

5 结论

本文建立了 GFlowNet 与熵正则化强化学习之间的直接联系，并通过实验验证了这一点。如果固定 GFlowNet 的反向策略，适当地从 GFlowNet 有向无环图构造马尔可夫决策过程，并为所有转换设置特定奖励，我们证明了学习 GFlowNet 的前向策略 P_F 等价于学习

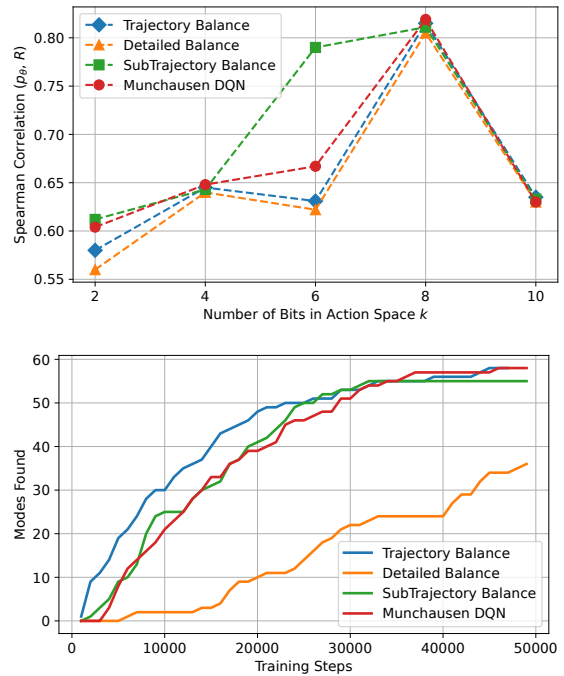


图 3：比特序列生成结果。上：每种方法在测试集上的 R 与 P_θ 之间的 Spearman 相关性，以及变化的 $k \in \{2, 4, 6, 8, 10\}$ 。下：训练过程中发现的模式数量对于 $k = 8$ 。

获得最优 RL 策略 π^* 当熵正则化系数 λ 设置为 1 时。在我们的实验中，我们展示了 M-DQN (Vieillard 等, 2020b) 与已有的 GFlowNet 训练方法 (Malkin 等, 2022; Bengio 等, 2023; Madan 等, 2023) 相比具有竞争力的结果，并且在某些情况下甚至优于它们。

进一步研究的一个有前景的方向是通过扩展文和范罗伊 (Wen 和 Van Roy, 2017) 的分析来探讨 GFlowNets 的理论性质。此外，将已建立的联系推广到离散设置之外也是有价值的，可以参考拉卢等人 (Lahlou 等人, 2023a) 的工作。从实用的角度来看，将蒙特卡洛树搜索 (MCTS) 算法扩展到 GFlowNets，如 AlphaZero (Silver 等人, 2018)，是一个有趣的探索方向，因为已知的确定性环境允许执行 MCTS 回放。最后，在现有的 GFlowNets 与 1) 生成建模 (Zhang 等人, 2022a)，2) 马尔可夫链蒙特卡洛方法 (Deleu 和 Bengio, 2023)，3) 变分推断 (Zimmermann 等人, 2022; Malkin 等人, 2023) 之间的联系背景下，加上本文提出的链接

4) 强化学习，另一个潜在的方向可以将 GFlowNets 视为不同机器学习领域和模型的统一视角，这可能导致建立有价值的联系。

致谢

第 4 节的实验结果由尼基塔·莫罗佐夫和亚历克谢·纳乌莫夫获得，得到了俄罗斯联邦政府分析中心（ACRF）提供的 AI 研究机构资助的支持，依据补贴提供协议（协议标识号 000000D730321P5Q0002）以及与 HSE 大学的协议编号 70-2021-00139。第 3 节的理论结果由丹尼尔·蒂亚普金在巴黎大区 DIM AI4IDF 框架下获得支持。本研究部分得到了 HSE 大学高性能计算设施的计算机资源支持（科斯特内茨基等，221 年）。

参考文献

- Bellemare, M., Srinivasan, S., Ostrovski, G., Schaul, T., Saxton, D., and Munos, R. (2016). Unifying count-based exploration and intrinsic motivation. *Advances in neural information processing systems*, 29.
- Bellemare, M. G., Dabney, W., and Rowland, M. (2023). *Distributional Reinforcement Learning*. MIT Press. <http://www.distributional-rl.org>.
- Bengio, E., Jain, M., Korablyov, M., Precup, D., and Bengio, Y. (2021). Flow network based generative models for non-iterative diverse candidate generation. *Advances in Neural Information Processing Systems*, 34:27381–27394.
- Bengio, Y., Lahlou, S., Deleu, T., Hu, E. J., Tiwari, M., and Bengio, E. (2023). Gflownet foundations. *Journal of Machine Learning Research*, 24(210):1–55.
- Bou, A., Bettini, M., Dittert, S., Kumar, V., Sodhani, S., Yang, X., De Fabritiis, G., and Moens, V. (2023). Torchrl: A data-driven decision-making library for pytorch. *arXiv preprint arXiv:2306.00577*.
- Burda, Y., Edwards, H., Storkey, A., and Klimov, O. (2019). Exploration by random network distillation. In *International Conference on Learning Representations*.
- Christodoulou, P. (2019). Soft actor-critic for discrete action settings. *arXiv preprint arXiv:1910.07207*.
- Cover, T. M. (1999). *Elements of information theory*. John Wiley & Sons.
- Deleu, T. and Bengio, Y. (2023). Generative flow networks: a markov chain perspective. *arXiv preprint arXiv:2307.01422*.
- Deleu, T., Góis, A., Emezue, C., Rankawat, M., Lacoste-Julien, S., Bauer, S., and Bengio, Y. (2022). Bayesian structure learning with generative flow networks. In *Uncertainty in Artificial Intelligence*, pages 518–528. PMLR.
- Fox, R., Pakman, A., and Tishby, N. (2016). Taming the noise in reinforcement learning via soft updates. In Ihler, A. and Janzing, D., editors, *Proceedings of the Thirty-Second Conference on Uncertainty in Artificial Intelligence, UAI 2016, June 25-29, 2016, New York City, NY, USA*. AUAI Press.
- Geist, M., Scherrer, B., and Pietquin, O. (2019). A theory of regularized markov decision processes. In *International Conference on Machine Learning*, pages 2160–2169. PMLR.
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. (2017). Neural message passing for quantum chemistry. In *International conference on machine learning*, pages 1263–1272. PMLR.
- Haarnoja, T., Tang, H., Abbeel, P., and Levine, S. (2017). Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pages 1352–1361. PMLR.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Dy, J. and Krause, A., editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR.
- Huang, S., Dossa, R. F. J., Ye, C., Braga, J., Chakraborty, D., Mehta, K., and Araújo, J. G. (2022). Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18.
- Jin, W., Barzilay, R., and Jaakkola, T. (2020). Junction tree variational autoencoder for molecular graph generation. In *Artificial Intelligence in Drug Discovery*, pages 228–249. The Royal Society of Chemistry.
- Kostenetskiy, P., Chulkevich, R., and Kozyrev, V. (2021). Hpc resources of the higher school of economics. In *Journal of Physics: Conference Series*, volume 1740, page 012050. IOP Publishing.
- Lahlou, S., Deleu, T., Lemos, P., Zhang, D., Volokhova, A., Hernández-García, A., Ezzine, L. N., Bengio, Y., and Malkin, N. (2023a). A theory of continuous generative flow networks. In *International Conference on Machine Learning*, pages 18269–18300. PMLR.
- Lahlou, S., Viviano, J. D., and Schmidt, V. (2023b). torchgfn: A pytorch gflownet library. *arXiv preprint arXiv:2305.14594*.
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. (2015).

- Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*.
- Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*, 8:293–321.
- Madan, K., Rector-Brooks, J., Korablyov, M., Bengio, E., Jain, M., Nica, A. C., Bosc, T., Bengio, Y., and Malkin, N. (2023). Learning gflownets from partial episodes for improved convergence and stability. In *International Conference on Machine Learning*, pages 23467–23483. PMLR.
- Malkin, N., Jain, M., Bengio, E., Sun, C., and Bengio, Y. (2022). Trajectory balance: Improved credit assignment in gflownets. *Advances in Neural Information Processing Systems*, 35:5955–5967.
- Malkin, N., Lahlou, S., Deleu, T., Ji, X., Hu, E. J., Everett, K. E., Zhang, D., and Bengio, Y. (2023). GFlownets and variational inference. In *The Eleventh International Conference on Learning Representations*.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533.
- Neu, G., Jonsson, A., and Gómez, V. (2017). A unified view of entropy-regularized markov decision processes. *arXiv preprint arXiv:1705.07798*.
- Osband, I., Blundell, C., Pritzel, A., and Van Roy, B. (2016). Deep exploration via bootstrapped dqn. *Advances in neural information processing systems*, 29.
- Ostrovski, G., Bellemare, M. G., Oord, A., and Munos, R. (2017). Count-based exploration with neural density models. In *International conference on machine learning*, pages 2721–2730. PMLR.
- Pan, L., Zhang, D., Courville, A., Huang, L., and Bengio, Y. (2023). Generative augmented flow networks. In *The Eleventh International Conference on Learning Representations*.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32.
- Rector-Brooks, J., Madan, K., Jain, M., Korablyov, M., Liu, C.-H., Chandar, S., Malkin, N., and Bengio, Y. (2023). Thompson sampling for improved exploration in gflownets. *arXiv preprint arXiv:2306.17693*.
- Saade, A., Kapturowski, S., Calandriello, D., Blundell, C., Sprechmann, P., Sarra, L., Groth, O., Valko, M., and Piot, B. (2023). Unlocking the power of representations in long-term novelty-based exploration. *arXiv preprint arXiv:2305.01521*.
- Schaul, T., Quan, J., Antonoglou, I., and Silver, D. (2016). Prioritized experience replay. In Bengio, Y. and LeCun, Y., editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*.
- Schulman, J., Chen, X., and Abbeel, P. (2017a). Equivalence between policy gradients and soft q-learning. *arXiv preprint arXiv:1704.06440*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017b). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Shen, M. W., Bengio, E., Hajiramezanali, E., Loukas, A., Cho, K., and Biancalani, T. (2023). Towards understanding and improving gflownet training. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144.
- Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., and Riedmiller, M. (2014). Deterministic policy gradient algorithms. In *International conference on machine learning*, pages 387–395. Pmlr.
- Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. *Machine learning*, 3:9–44.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.
- Tang, Y., Munos, R., Rowland, M., and Valko, M. (2023). VA-learning as a more efficient alternative to q-learning. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J., editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 33739–33757. PMLR.
- Tiapkin, D., Belomestny, D., Calandriello, D., Moulines, E., Munos, R., Naumov, A., Perrault, P., Tang, Y., Valko, M., and Ménard, P. (2023). Fast rates for maximum entropy exploration. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org.
- Trott, O. and Olson, A. J. (2010). Autodock vina: improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of computational chemistry*, 31(2):455–461.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Vemgal, N., Lau, E., and Precup, D. (2023). An empirical study of the effectiveness of using a replay buffer on mode discovery in gflownets. *arXiv preprint arXiv:2307.07674*.
- Vieillard, N., Kozuno, T., Scherrer, B., Pietquin, O., Munos, R., and Geist, M. (2020a). Leverage the average: an analysis of kl regularization in reinforcement learning. *Advances in Neural Information Processing Systems*, 33:12163–12174.
- Vieillard, N., Pietquin, O., and Geist, M. (2020b). Munchausen reinforcement learning. *Advances in Neural Information Processing Systems*, 33:4235–4246.
- Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., and Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR.
- Wen, Z. and Van Roy, B. (2017). Efficient reinforcement learning in deterministic systems with value function generalization. *Mathematics of Operations Research*, 42(3):762–782.
- Zahavy, T., O’Donoghue, B., Desjardins, G., and Singh, S. (2021). Reward is enough for convex mdps. *Advances in Neural Information Processing Systems*, 34:25746–25759.
- Zhang, D., Chen, R. T., Malkin, N., and Bengio, Y. (2022a). Unifying generative models with gflownets. *arXiv preprint arXiv:2209.02606*.
- Zhang, D., Malkin, N., Liu, Z., Volokhova, A., Courville, A., and Bengio, Y. (2022b). Generative flow networks for discrete probabilistic modeling. In *International Conference on Machine Learning*, pages 26412–26428. PMLR.
- Zhang, D., Pan, L., Chen, R. T., Courville, A., and Bengio, Y. (2023). Distributional gflownets with quantile flows. *arXiv preprint arXiv:2302.05793*.
- Zimmermann, H., Lindsten, F., van de Meent, J.-W., and Naesseth, C. A. (2022). A variational perspective on generative flow networks. *arXiv preprint arXiv:2210.07992*.

检查表

1. 对于所有提出的模型和算法，确认是否包含：(a) 数学设定、假设、算法和/或模型的清晰描述。[是，请参见第 2 节和第 3 节，以及附录 C]

(b) 对任何算法的性质和复杂度（时间、空间、样本量）的分析。[是，请参见附录 C]

(c)（可选）匿名源代码，包括所有依赖项的说明，如外部库。[是，请参见第 1 节]

2. 对于所有理论声明，确认是否包含：

(a) 所有理论结果的完整假设集的陈述。[是，请参见第 3 节]

(b) 所有理论结果的完整证明。[是，请参见附录 B]

(c) 对任何假设的清晰解释。[是，请参见第 3 节]

3. 对于所有展示实证结果的图表，确认是否包含：

(a) 复现主要实验结果所需的代码、数据和说明（在补充材料或 URL 中提供）。[是，请参见第 1 节和附录 D] (b) 所有训练细节（例如，数据划分、超参数及其选择方式）。[是，请参见第 4 节和附录 D] (c) 具体测度或统计量及其误差条的清晰定义（例如，多次运行实验后关于随机种子的误差条）。[是，请参见第 4 节和附录 D] (d) 所使用的计算基础设施的描述（例如，GPU 类型、内部集群或云服务提供商）。[是，请参见附录 D]

4. 若您正在使用现有资源（例如代码、数据、模型）或整理/发布新资源，请确认是否包含：

(a) 创作者的引用 如果您的作品使用了现有资源。[是，请参见附录 D] (b) 资源的许可证信息，如适用。[不适用] (c) 新资源，如有，要么在补充材料中，要么提供一个 URL。[不适用] (d) 数据提供者/管理员的同意信息。[不适用] (e) 如有适用，讨论合理的内容，例如个人身份信息或冒犯性内容。[不适用]

5. 若您使用了众包服务或对人类受试者进行了研究，请确认是否包含：(a) 给参与者的完整指令文本和截图。[不适用]

(b) 参与者潜在风险的说明，如有必要，请附上机构审查委员会（IRB）的批准链接。[不适用] (c) 支付给参与者的每小时工资估算及参与者补偿总额。[不适用]

一种表示法

表 1: 文中使用的符号表。符号 含义

Notation	Meaning
$\mathcal{S}$	state space
$\mathcal{A}$	action space
$\mathcal{X}$	sampling space
$\mathcal{R}$	GFlowNet reward function
$\mathcal{F}(\tau), \mathcal{F}(s), \mathcal{F}(s \rightarrow s')$	flow function
$\mathcal{P}(\tau)$	distribution over trajectories induced by a flow
$\mathcal{P}(x)$	distribution over terminal states induced by a flow
$\mathcal{P}_F(s' s)$	GFlowNet forward policy
$\mathcal{P}_B(s s')$	GFlowNet backward policy
$Z, Z_{\mathcal{F}}$	normalizing constant of the reward distribution and a flow $\mathcal{F}$
$P(s' s, a)$	Markovian transition kernel for MDP
$r(s, a)$	MDP reward function
λ	regularization coefficient
$V_{\lambda}^{\pi}(s), Q_{\lambda}^{\pi}(s, a)$	regularized value and Q-value of a given policy π
$V_{\lambda}^{\star}(s), Q_{\lambda}^{\star}(s, a)$	regularized value and Q-value of an optimal policy
$V_1^{\pi}(s), Q_1^{\pi}(s, a)$	regularized value and Q-value of a given policy π with coefficient 1
$V_1^{\star}(s), Q_1^{\star}(s, a)$	regularized value and Q-value of an optimal policy with coefficient 1
$\pi_{\lambda}^{\star}(a s)$	regularized optimal policy
$q^{\pi}(\tau)$	distribution over trajecories induced by a policy π
$d^{\pi}(x)$	distribution over terminal states induced by a policy π
Softmax_{λ}	softmax function with a temperature $\lambda > 0$, for $x \in \mathbb{R}^d$ $[\text{Softmax}_{\lambda}(x)]_i \triangleq e^{x_i/\lambda} / (\sum_{j=1}^d e^{x_j/\lambda}) \quad \forall i \in [d]$
$\text{LogSumExp}_{\lambda}$	log-sum-exp function with a temperature $\lambda > 0$, for $x \in \mathbb{R}^d$ $\text{LogSumExp}(x) \triangleq \lambda \log(\sum_{i=1}^d e^{x_i/\lambda})$.
$\mathcal{H}$	Shannon entropy function, $\mathcal{H}(p) \triangleq \sum_{i=1}^n p_i \log(1/p_i)$.

设 $(X, \mathcal{X})$ 为可测空间, $\mathcal{P}(X)$ 表示该空间上所有概率测度的集合。对于 $p \in \mathcal{P}(X)$, 我们用 $\mathbb{E}_p$ 表示关于 p 的期望。对于随机变量 $\xi: X \rightarrow \mathbb{K}$, 记号 $\xi \sim p$ 表示 $\text{Law}(\xi) = p$. 对于任意 $p, q \in \mathcal{P}(X)$, Kullback-Leibler 发散 $\text{KL}(p, q)$ 由下式给出

$$\text{KL}(p, q) \triangleq \begin{cases} \mathbb{E}_p \left[\log \frac{dp}{dq} \right], & p \ll q, \\ +\infty, & \text{otherwise.} \end{cases}$$

B 缺失的证明

B.1 定理 1 的证明

Let $\mathcal{F}: \mathcal{T} \rightarrow \mathbb{R}_{\geq 0}$ be a Markovian flow uniquely defined by a backward policy $\mathcal{P}_B$ and a GFlowNet reward $\mathcal{R}$ that satisfies reward matching constraint (1). By soft Bellman equations (8), the value and the Q-value of the optimal policy satisfy for all $s \in \mathcal{S} \setminus \mathcal{X}, s' \in \mathcal{A}_s$

$$\begin{aligned} Q_1^{\star}(s, s') &= r(s, s') + V_1^{\star}(s'), \\ V_1^{\star}(s) &= \log \left(\sum_{s' \in \mathcal{A}_s} \exp(Q_1^{\star}(s, s')) \right) \end{aligned} \quad (13)$$

并且, 另外地, $V_1^{\star}(s_f) = 0$ 及 $V_1^{\star}(x) = \log \mathcal{R}(x)$ for any $x \in \mathcal{X}$.

Next, we claim that for any $s, s' \in \mathcal{S}$ it holds $V_1^{\star}(s) = \log \mathcal{F}(s)$, $Q_1^{\star}(s, s') = \log \mathcal{F}(s \rightarrow s')$ and prove it by backward induction over the topological ordering of vertices of $\mathcal{G}$. The base of induction already holds for $V_1^{\star}(x)$

对于所有 $x \in \mathcal{X}$ 。令 $s \in \mathcal{S} \setminus \mathcal{X}$ 为一个固定状态，并假设对于其所有子节点 $s' \in \mathcal{A}_s$ 该命题已得证。根据软贝尔曼方程 (13)，对于任意 $s' \in \mathcal{A}_s$ ，我们有

$$\begin{aligned} Q_1^*(s, s') &= \log \mathcal{P}_B(s|s') + V_1^*(s') \\ &= \log \frac{\mathcal{F}(s \rightarrow s')}{\mathcal{F}(s')} + \log \mathcal{F}(s') = \log \mathcal{F}(s \rightarrow s'), \end{aligned}$$

其中我们使用了 MDP 奖励的选择 (11) 以及通过流函数 (2) 定义的 $\mathcal{P}_B$ 与归纳假设相结合的结果。接下来，软贝尔曼方程表明

$$\exp(V_1^*(s)) = \sum_{s' \in \mathcal{A}_s} \exp(Q_1^*(s, s')) = \sum_{s' \in \mathcal{A}_s} \mathcal{F}(s \rightarrow s') = \mathcal{F}(s),$$

其中应用了 Q 值的归纳假设和流加工约束 (Bengio 等, 2023 年, 命题 19)。最后，利用 (9)，我们得到最优策略的如下表达式

$$\pi_1^*(s'|s) = \exp(Q_1^*(s, s') - V_1^*(s)) = \frac{\mathcal{F}(s \rightarrow s')}{\mathcal{F}(s)}$$

这与对应的流的前向策略 $\mathcal{P}_F$ (2) 一致。

B.2 命题 1 的证明

令 N 等于轨迹最大长度 $\max_{\tau} n_{\tau}$ ，即在步骤 N 智能体保证到达具有零奖励的吸收状态。然后，根据定义 (7) 的 $V_1^{\pi}(s_0)$ 在正则化马尔可夫决策过程中的 $\gamma = \lambda = 1$ 我们有

$$V_1^{\pi}(s_0) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{N-1} r(s_t, s_{t+1}) - \log \pi(s_{t+1}|s_t) \right],$$

其中我们利用了条件期望的塔性质来将熵替换为负对数。通过代入奖励的表达式，我们得到

$$\begin{aligned} V_1^{\pi}(s_0) &= \mathbb{E}_{\tau \sim q^{\pi}} \left[\sum_{t=1}^{n_{\tau}} \log \frac{\mathcal{P}_B(s_{t-1}|s_t)}{\pi(s_t|s_{t-1})} + \log \mathcal{R}(s_{n_{\tau}}) \right] \\ &= \mathbb{E}_{\tau \sim q^{\pi}} \left[\log \frac{\prod_{t=1}^{n_{\tau}} \mathcal{P}_B(s_{t-1}|s_t) \cdot \mathcal{R}(s_{n_{\tau}}) \cdot Z}{\prod_{t=1}^{n_{\tau}} \pi(s_t|s_{t-1}) \cdot Z} \right] = -\text{KL}(q^{\pi} \parallel \mathcal{P}_B) + \log Z. \end{aligned}$$

附录 C 算法详细描述

本节提供了 SoftDQN 和 MunchausenDQN 的详细算法描述，并讨论了几种超参数的影响。

C.1 Soft DQN

该算法的流程如下。在学习过程开始时，我们有两个神经网络： Q_{θ} 和 $Q_{\bar{\theta}}$ ，以及 $\bar{\theta} = \theta$ 。在算法的每次迭代中，通过遵循 Softmax 策略 $\pi_{\theta}(s) = \text{Softmax}_{\lambda}(Q_{\theta}(s))$ （此处 $\lambda = 1$ ）使用在线网络 Q_{θ} （可选地）附加 ε -贪婪探索来采样若干轨迹（通常为 4 或 16）。

¹

¹ 值得注意的是，Softmax 参数化自然诱导 Boltzmann 探索。然而，Boltzmann 和 ε -探索的效果不同，一般不能互相替代，详见 (Vieillard 等, 2200b) 中的讨论。

Algorithm 1 SoftDQN (Haarnoja et al., 2017) / MunchausenDQN (Vieillard et al., 2020b)

- 1: **Input:** trajectory budget N , number of trajectories on one update M , exploration parameter ε , backward kernel $\mathcal{P}_B$, PER $\mathcal{B}$, PER batch size B , hard update parameter T , soft update parameter τ , Munchausen parameters α, l_0 ;
- 2: Initialize parameters of online network θ and target network $\bar{\theta} = \theta$;
- 3: Define entropy coefficient $\lambda = 1/(1 - \alpha)$
- 4: **for** $t = 1, \dots, \lceil N/M \rceil$ **do**
- 5: Sample M trajectories interaction with ε -greedy version of policy $\pi_\theta = \text{Softmax}_\lambda(Q_\theta)$, and place them into replay buffer $\mathcal{B}$;
- 6: Sample B transitions $\{(s_k, a_k, R_k, s'_k, d_k)\}_{k=1}^B$ from PER $\mathcal{B}$, where d_k is a DONE flag: $d_k = \mathbb{1}\{s_k \in \mathcal{X}\}$, and R_k is GflowNet reward is $d_k = 1$ and 0 otherwise;
- 7: Compute MDP rewards: $r_k = d_k \log R_k + (1 - d_k) \log \mathcal{P}_B(s_k | s'_k)$ with a convention $0 \log 0 = 0$ for all $k \in \{1, \dots, B\}$.
- 8: Compute TD targets

$$y_k = r_k + (1 - d_k) \text{LogSumExp}_\lambda(Q_{\bar{\theta}}(s'_k)) + \alpha \max\{\lambda \log \pi_{\bar{\theta}}(a_k | s_k), l_0\}$$

optionally using a convention $\text{LogSumExp}_\lambda(Q_{\bar{\theta}}(s'_k)) = \log \mathcal{R}(s'_k) \forall s'_k \in \mathcal{X}$ (see explanation below);

- 9: Use y_k to update priorities in PER (Schaul et al., 2016) as

$$p_k = \ell(Q_\theta(s_k, a_k), y_k),$$

where ℓ is a Huber loss (15).

- 10: Compute TD loss

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{k=1}^B \ell(Q_\theta(s_k, a_k), y_k),$$

where ℓ is a Huber loss (15).

- 11: Perform gradient update by $\nabla_\theta \mathcal{L}(\theta)$;
 - 12: **if** $t \bmod T = 0$ **then**
 - 13: Update target network $\bar{\theta} := (1 - \tau)\bar{\theta} + \tau\theta$;
 - 14: **end if**
 - 15: **end for**
 - 16: **Output** policy π_θ .
-

所有这些采样的轨迹被分割成转换 $(s_t, a_t, s_{t+1}, R_t, d_t)$ ，其中 $R_t = \log \mathcal{R}(s_t)$ 如果 $s_t \in \mathcal{X}$ 成立，则取值为 1，否则为 0， $d_t = \mathbb{1}\{s_t \in \mathcal{X}\}$ is a DONE 浮点数。请注意，我们总是有 $a_t = s_{t+1}$ ，而 a_t 仅用于与常见的 RL 表示保持一致。这些分离的转换被放入带有优先级机制的回放缓冲区 $\mathcal{B}$ (Schaul 等, 2016)。

接下来，一批状态转移 $\{(s_k, a_k, s'_k, R_k, d_k)\}_{k=1}^B$ 从回放缓冲区中采样 $\mathcal{B}$ 通过优先级处理。MDP 奖励计算如下 $r_k = d_k \log \mathcal{R}(s_k) + (1 - d_k) \log \mathcal{P}_B(s_k | s'_k)$ ，TD 目标值按以下方式计算：

$$\forall k \in [B] : y_k = r_k + (1 - d_k) \text{LogSumExp}_\lambda(Q_{\bar{\theta}}(s'_k)), \quad (14)$$

where $\lambda = 1$.

另一种方法是完全跳过导致汇状态的转换 s_f (对于这些转换 $d_k = 1$)，而是替换为 $\text{LogSumExp}_\lambda(Q_{\bar{\theta}}(s'_k)) = \log \mathcal{R}(s'_k) \forall s'_k \in \mathcal{X}$ 。这是因为我们知道所有 $s \in \mathcal{X}$ 的 $V_\lambda^*(s)$ 的确切值。我们在实验中采用了这种选择。

给定目标后，我们更新回放缓冲区中采样批次的优先级，并按如下方式计算回归 TD 损失：

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{k=1}^B \ell(Q_\theta(s_k, a_k), y_k),$$

where ℓ is a Huber loss

$$\ell(x, y) = \begin{cases} \frac{1}{2}(x - y)^2 & |x - y| \leq 1 \\ |x - y| - 1/2 & |x - y| \geq 1. \end{cases} \quad (15)$$

最后, 在每次迭代结束时, 我们执行硬更新, 即将在线网络的权重复制到目标网络的权重, 遵循原始 DQN 方法 (Mnih 等, 2015), 或者执行软更新, 即应用 Polyak 平均, 参数为 $\tau < 1$, 遵循 DDPG 方法 (Silver 等, 2014; Lillicrap 等, 2015)。

该算法的详细描述也提供在算法 1 中。请注意, 该算法的复杂度与采样的转换数量成线性关系, 因为缓冲区的大小在训练过程中是固定的。

接下来, 我们将更详细地讨论此流水线的一些元素。

优先经验回放 优先经验回放的本质 (Schaul 等, 2016) 是从回放缓冲区中根据以下计算的优先级进行非均匀采样:

$$P(i) = \frac{|p_i|^\alpha}{\sum_{i \in \mathcal{B}} |p_i|^\alpha},$$

其中 $P(i)$ 是从回放缓冲区采样第 i 个转换的概率, $\mathcal{B}$, p_i 是等于该转换最后计算的 TD 误差的优先级, 而 α 是一个温度参数, 用于在均匀采样 $\alpha = 0$ 和纯粹优先级采样 $\alpha = +\infty$ 之间进行插值。通常选择的 α 值介于 0.4 和 0.6 之间, 然而, 在两个实验 (分子生成和比特序列生成) 中, 我们发现选择一个激进的值 $\alpha = 0.9$ 来更快地收集困难样本更有利。

此外, (Schaul 等, 2016) 的作者应用了重要性采样 (IS) 校正, 因为理论要求在缓冲区上进行均匀采样, 由于下一个状态的随机性 s'_k 。平滑校正由系数 β 控制, 该系数应在训练过程中逐渐增加到 1。

然而, 在 GFlowNets 的设置中, 我们注意到这种校正是不必要的, 因为下一个转换和奖励总是确定性的; 因此可以将 β 设置为一个小常数而不损失收敛特性。我们在超网格实验中设置了 $\beta = 0.0$, 在分子和比特序列生成实验中设置了 $\beta = 0.1$, 并发现这比进行更高的 IS 校正更有益。

损失函数的选择 我们考察了两种类型的回归损失: 经典的均方误差 (MSE) 损失和 Huber 损失, 后者在 DQN (Mnih 等, 2015) 和 MunchausenDQN (Vieillard 等, 2020b) 中被应用。Huber 损失的主要思想是自动梯度裁剪对于非常大的更新。我们观察到使用 Huber 损失时训练更加稳定的好处, 因此我们在所有实验中都使用了它, 除了附录 E 中的一些消融研究。

附录 C.2 Munchausen DQN

GFlowNets 上下文中 SoftDQN 与 M-DQN 的区别包括:

- 两个额外的参数: α 和截断参数 l_0 ;
- 使用熵系数 λ 等于 $1/(1 - \alpha)$ 而非仅仅 $\lambda = 1$ 。特别是它改变了策略的方式 $\pi_\theta(s) = \text{Softmax}_{1/(1 - \alpha)}(Q_\theta(s))$ 以及值 $V_\theta(s) = \text{对数和指数}(\text{LogSumExp})_{1/(1 - \alpha)}(Q_\theta(s))$ 被计算;
- 额外项 $\alpha \max\{\lambda \log \pi_\theta(a_k|s_k), l_0\}$ 在计算 TD 目标值 (14) 的过程中。

所有这些差异在算法 1 中以蓝色突出显示。使用不同的熵系数由 Vieillard 等 (2020b) 的定理 1 所证明: 带有熵系数 λ 和蒙豪森系数 α 的 M-DQN 等价于求解具有系数 $(1 - \alpha)\lambda$ 的熵正则化强化学习问题。

D 实验细节

在本节中, 我们将提供第 4 节实验的更多细节。

D.1 超网格

在 $s^\top = (s^1, \dots, s^D)^\top$ 处奖励的形式定义为

$$\begin{aligned} \mathcal{R}(s^\top) = & R_0 + R_1 \prod_{i=1}^D \mathbb{I} \left[0.25 < \left| \frac{s^i}{H-1} - 0.5 \right| \right] \\ & + R_2 \prod_{i=1}^D \mathbb{I} \left[0.3 < \left| \frac{s^i}{H-1} - 0.5 \right| < 0.4 \right], \end{aligned}$$

在 $0 < R_0 \ll R_1 < R_2$ 处。奖励函数在网格的角落附近有峰值，这些峰值之间由奖励极低的宽阔凹槽隔开，具体数值为 R_0 。在第 4 节中，我们使用了 Malkin 等人 (2022) 提出的奖励参数值 ($R_0 = 10^{-3}, R_1 = 0.5, R_2 = 2.0$)。

所有模型均通过与 Bengio 等人 (2021) 相同的多层感知机架构进行参数化，包含两层隐藏层，每层隐藏层大小为 256。遵循 Malkin 等人 (2022) 的方法，我们使用 Adam 优化器训练所有模型，并设置学习率为 10^{-3} ，批大小为 16（每次训练步骤中采样的轨迹数）。对于 TB 情况，我们按照 Malkin 等人 (2022) 的方法，将学习率设为 10^{-1} ，持续时间设为 Z_θ 。对于 SubTB 参数 λ （不应与软 RL 正则化系数混淆），我们根据 Madan 等人 (2023) 的方法将其值设为 0.9。所有模型均训练至采样到 10^6 条轨迹为止，经验样本分布 $\pi(x)$ 基于训练过程中最后见到的 $2 \cdot 10^5$ 个样本计算得出，这解释了在图 1 中在 $2 \cdot 10^5$ 处出现的下降现象。

对于 M-DQN，我们采用了 Huber 损失而非均方误差损失，遵循 Mnih 等人 (2015) 和 Vieillard 等人 (2020b) 的方法，并使用了优先经验回放缓冲区 (Schaul 等人, 2016)，该缓冲区来自 TorchRL 库 (Bou 等人, 2023)，其大小为 100,000，标准常数为 $\alpha = 0.5$ ，并且不进行重要性采样修正 $\beta = 0$ 。在每次训练步骤中，我们抽取 16 条轨迹，将所有转换放入缓冲区，然后从缓冲区中抽取 256 个转换来计算损失。因此，M-DQN 使用与基线相同的轨迹数量和梯度步数进行训练。关于具体的 Munchhausen 参数 (Vieillard 等人, 2020b)，我们使用 $\alpha = 0.15$ ，以及异常大的 $l_0 = -100$ ，因为我们的设置中的熵系数远大于原始 M-DQN 中使用的值。为了更新目标网络，我们采用了类似于著名 DDPG (Silver 等人, 2014; Lillicrap 等人, 2015) 的软更新方法，由于在获取批次之前抽取了 16 条轨迹，因此使用较大的系数 $\tau = 0.25$ 。

对于超网格环境实验，我们使用了 torchgfn 库 (Lahlou 等人, 2023b)。所有实验均在 CPU 上执行。

D.2 分子

所有模型均通过 MPNN (Gilmer 等人, 2017) 参数化，我们使用了与先前工作 (Bengio 等人, 2021; Malkin 等人, 2022; Madan 等人, 2023) 相同的参数。奖励代理和测试集也取自 Bengio 等人 (2021)。根据 Madan 等人 (2023)，我们考虑奖励指数从 $\{4, 8, 10, 16\}$ ，学习率从 $\{5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}, 10^{-3}\}$ ，并使用 Adam 优化器对所有模型进行 50,000 次迭代的训练，批大小为 4。对于 SubTB，我们使用与 Madan 等人 (2023) 相同的值 $\lambda = 1.0$ 。图 2 中基线的奖励相关性直接取自 Madan 等人 (2023)。我们跟踪 Tanimoto 分离模式，如 Bengio 等人 (2021) 所述，使用原始奖励阈值为 7.0，Tanimoto 相似性阈值为 0.7。

关于 M-DQN，我们采用了 Huber 损失而非均方误差，遵循 Mnih 等人 (2015) 和 Vieillard 等人 (2020b) 的方法，并发现利用对战架构 (Wang 等人, 2016) 带来了显著的好处，这与 DB 构建了相似性 (参见第 3.4 节)。我们还利用了优先经验回放缓冲区 (Schaul 等人, 2016)，该缓冲区来自 TorchRL 库 (Bou 等人, 2023)，其大小为 1,000,000，并使用了不寻常的参数 $\alpha = 0.9, \beta = 0.1$ （有关此选择的详细信息，请参阅附录 C），以及 Munchhausen 参数 $\alpha = 0.15$ 和 $l_0 = -2500$ 。在每次训练步骤中，我们采样 4 条轨迹，将所有转换放入缓冲区，然后从缓冲区中采样 256 个转换来计算损失。为了更新目标网络，我们采用了类似于 DDPG (Silver 等人, 2014; Lillicrap 等人, 2015) 的软更新方法，系数为 $\tau = 0.1$ 。请注意，M-DQN 用于训练的轨迹数量与基线完全相同，因此在发现模式方面的比较是公平的。

我们的实验基于 Bengio 等人 (2021) 和 Malkin 等人 (2022) 发布的代码。我们利用配备有 NVIDIA V100 和 NVIDIA A100 GPU 的集群进行分子生成实验。

D.3 二进制序列

模式集合 M 和测试集的构建方式如 Malkin 等人 (2022) 所述。我们也使用相同的变换器 (Vaswani 等人, 2017) 神经网络架构, 所有模型具有 3 个隐藏层, 64 个隐藏维度和 8 个注意力头, 唯一的区别在于它输出更大的动作空间的对数概率, 因为我们生成序列是非自回归的。

为了近似 $\mathcal{P}_\theta(x)$, 我们采用了 Zhang 等人 (2022b) 提出的蒙特卡洛估计方法:

$$\mathcal{P}(x) = \mathbb{E}_{\mathcal{P}_B(\tau|x)} \frac{\mathcal{P}_F(\tau)}{\mathcal{P}_B(\tau|x)} \approx \frac{1}{N} \sum_{i=1}^N \frac{\mathcal{P}_F(\tau^i)}{\mathcal{P}_B(\tau^i|x)},$$

其中 $\mathcal{P}_F(\tau) = \prod_{t=1}^{\tau} \mathcal{P}_F(s_t|s_{t-1}, \theta)$, $\mathcal{P}_B(\tau|x) = \prod_{t=1}^{\tau} \mathcal{P}_B(s_t|s_{t-1})$ 并且 $\tau^i \sim \mathcal{P}_B(\tau|x)$. Zhang 等人 (2022b) 表明, 即使在 $N = 10$ 的情况下, 这种方法也能提供一个充分的估计, 因此我们在实验中使用这个值。

我们对所有模型进行了 50,000 次迭代的训练, 批大小为 16, 使用 Adam 优化器。对于所有方法, 轨迹的采样使用了 $\mathcal{P}_F$ 和下一个可能动作的均匀分布的混合, 后者具有权重 10^{-3} (ϵ (贪婪探索))。我们将奖励指数值设置为 2。对于 M-DQN 和所有基线, 我们从 $\{5 \times 10^{-4}, 10^{-3}, 2 \times 10^{-3}\}$ 中选取最佳的学习率。对于 SubTB, 我们从 $\{0.9, 1.1, 1.5, 1.9\}$ 中选取最佳的 λ 。

对于 M-DQN, 我们采用了 Huber 损失而非均方误差, 参考了 (Mnih 等人, 2015; Vieillard 等人, 2020b), 并使用来自 TorchRL 库 (Bou 等人, 2023) 的优先级回放缓冲区, 其大小为 100,000, 以及常量 $\alpha = 0.9, \beta = 0.1$ 。在每次训练步骤中, 我们采样 16 条轨迹并将所有转换放入缓冲区, 然后从缓冲区中采样 256 个转换来计算损失。关于具体的 Munchausen 参数 (Vieillard 等人, 2020b), 我们使用 $\alpha = 0.15$ 和 $l_0 = -25$ 。目标网络仅使用硬更新 (Mnih 等人, 2015), 频率为每 5 次迭代一次。请注意, M-DQN 与基线使用的训练轨迹数量完全相同, 因此在发现模式方面的比较是公平的。此外, 我们发现在 M-DQN 中增加导向终止状态的边的损失权重是有帮助的, 这也在原始 GFlowNet 论文 (Bengio 等人, 2021) 中用于流匹配目标, 并在我们的实验中选取最佳权重系数从 $\{2, 5\}$ 。

对于比特序列实验, 我们使用了自己在 PyTorch(Paszke 等人, 2019) 上的代码实现。我们还利用了一个配备有 NVIDIA V100 和 NVIDIA A100 GPU 的集群。

附录 E 额外的比较

本节提供了额外的实验比较和消融研究。

附录 E.1 具有困难奖励的超网格

我们提供了 Madan 等人 (2023) 提出的具有更难奖励变体的超网格的实验结果, 其参数为 $(R_0 = 10^{-4}, R_1 = 1.0, R_2 = 3.0)$ 。我们遵循第 4 节中的相同实验设置。结果展示在图 4 中。我们做出了与第 4 节中所述相似的观察: 当 $\mathcal{P}_B$ 对所有方法固定为均匀分布时, M-DQN 比基线更快收敛, 而当 $\mathcal{P}_B$ 由基线学习时, M-DQN 显示出与 SubTB 相当的性能, 并优于 TB 和 DB。值得注意的是, TB 在大多数情况下无法收敛, 这与 Madan 等人 (2023) 的结果一致。

附录 E.2 训练反向策略

我们的理论框架不支持带有可学习反向策略的软 RL 运行, 因此在第 4 节中, 我们在所有情况下均使用了均匀的 M-DQN $\mathcal{P}_B$ 。尽管如此, 仍可以尝试使用相同的 M-DQN 训练损失来计算梯度并更新 $\mathcal{P}_B$, 从而创建一个具有可学习反向策略的设置。图 5 展示了在超网格环境中的比较结果 (使用标准奖励, 与第 4 节相同)。我们发现, 对于 M-DQN 而言, 具有可训练的 $\mathcal{P}_B$ 并不会提高其收敛速度, 这与之前提出的 GFlowNet 训练方法 (见图 1) 中学习到的 $\mathcal{P}_B$ 的行为相反。

另一种可能的方法是使用其他 GFlowNet 训练目标来计算损失值 $\mathcal{P}_B$, 例如 TB, 同时使用软 RL 来训练算法 $\mathcal{P}_F$ 。我们将这种情况留待进一步研究。

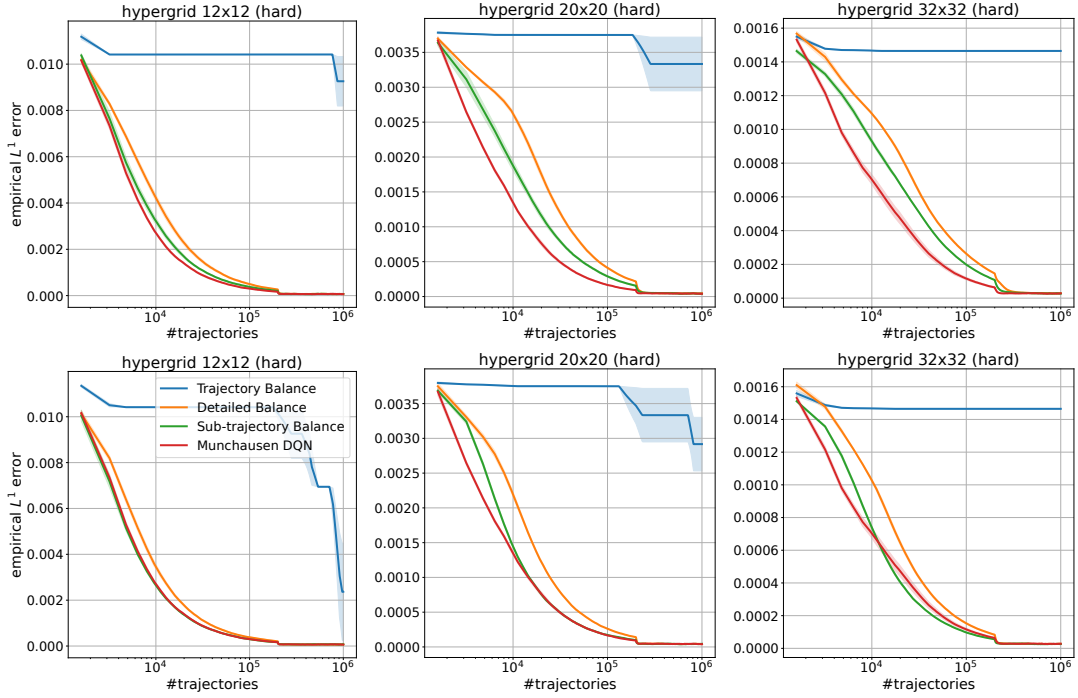


图 4: 在超网格环境中针对硬奖励变体进行训练期间, 目标和经验 GFlowNet 分布之间的距离 L^1 。上行: 对于所有方法, 反向策略 $\mathcal{P}_B$ 固定为均匀分布。下行: 基线方法中反向策略 $\mathcal{P}_B$ 是学习得到的, 而 M-DQN 则固定为均匀分布。

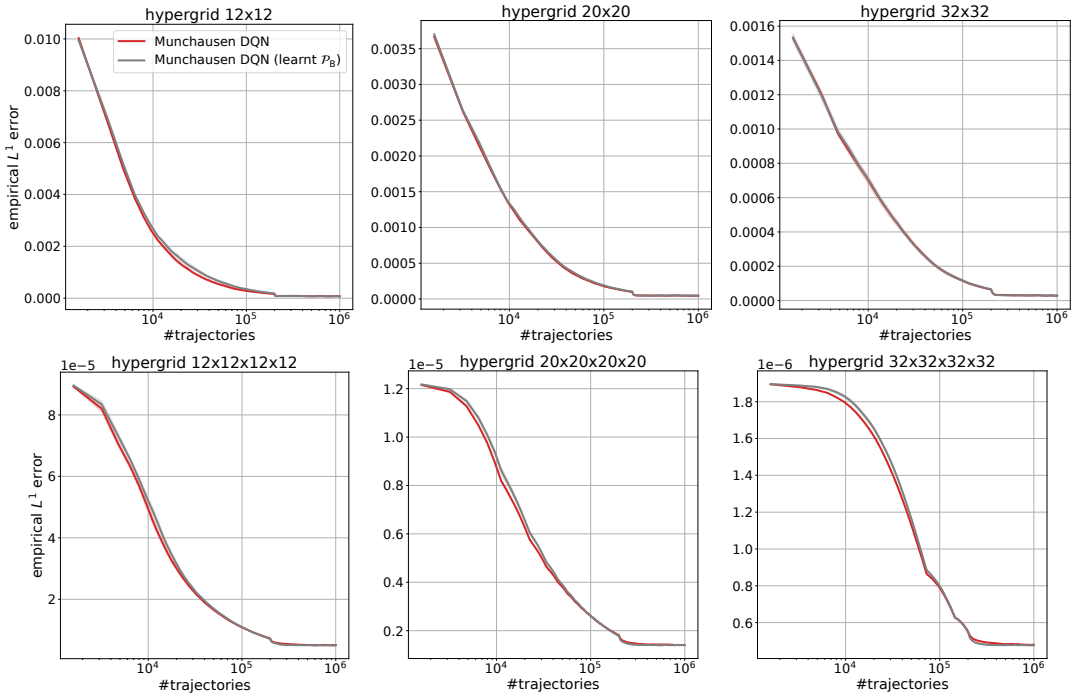


图 5: 在超网格环境中进行训练期间, 目标和经验 GFlowNet 分布之间的距离 L^1 。这里我们展示了两版本 M-DQN: 一种具有固定的均匀反向策略 $\mathcal{P}_B$, 另一种在训练过程中学习反向策略 $\mathcal{P}_B$ 。

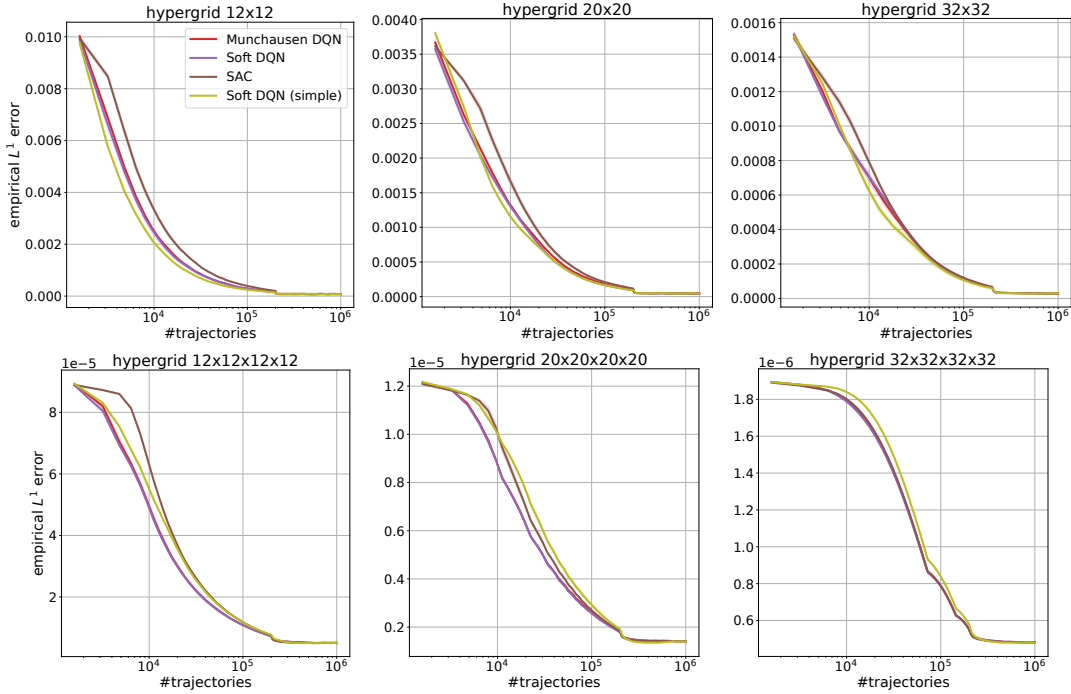


图 6: 在超网格环境中进行训练期间, 目标和经验 GFlowNet 分布之间的距离 L^1 。对于所有方法, 反向策略 $\mathcal{P}_B$ 固定为均匀分布。这里我们比较了不同的软 RL 算法。

附录 E.3 不同的软 RL 算法

附录 E.3.1 超网格环境

我们也比较了 M-DQN 与其他已知的离策略软 RL 算法:

软 DQN 为了消融 Munchausen 惩罚在 M-DQN 中的效果, 我们将它与经典版本的 SoftDQN 进行比较, 后者相当于将 Munchausen 常数设置为 $\alpha = 0$ 。所有其他超参数的选择与 M-DQN 完全相同 (参见附录 D)。

软 DQN (简化版) 为了消融使用回放缓冲区和 Huber 损失的效果, 我们也训练了一个简化的 SoftDQN 版本。在每次训练迭代中, 它会采样一批轨迹, 并使用这些采样的转换来计算损失, 类似于 GFlowNet 使用 DB 目标进行训练的方式。它还使用均方误差损失而不是 Huber 损失。

演员评论家 我们还考虑了 SAC (Haarnoja 等人, 2018) 算法, 即 Christodoulou (2019) 提出的离散版本。我们遵循 CleanRL 库 (Huang 等人, 2022) 中的实现, 有两个不同之处: 使用 Huber 损失 (Mnih 等人, 2015) 以及不调整熵正则化参数 (因为我们的理论规定将其值设为 1)。所有其他超参数的选择与 M-DQN 和 SoftDQN 完全相同 (参见附录 D)。

图 6 展示了在超网格环境 (具有标准奖励, 与第 4 节相同) 上的比较结果。我们发现 M-DQN 和 SoftDQN 通常具有非常相似的性能, 而在大多数情况下比 SAC 收敛得更快。有趣的是, 简化的 SoftDQN 版本在较小的二维网格上与其他算法竞争时表现良好, 收敛速度更快, 但在四维网格上落后于 M-DQN 和 SoftDQN。这表明软强化学习的成功不仅是因为使用了回放缓冲区和 Huber 损失而非均方误差损失, 而且其训练目标本身非常适合 GFlowNet 的学习问题。

E.3.2 小分子生成

我们还将 M-DQN 和 SoftDQN 在分子生成任务上进行了比较。实验设置与第 4 节相同。图 7 展示了结果。与 M-DQN 相比，SoftDQN 在 $\beta = 16$ 的情况下表现较差。但在其他所有情况下，它显示出相当或更好的奖励相关性，并且对学习率的选择更为鲁棒。然而，M-DQN 发现模式的速度更快。

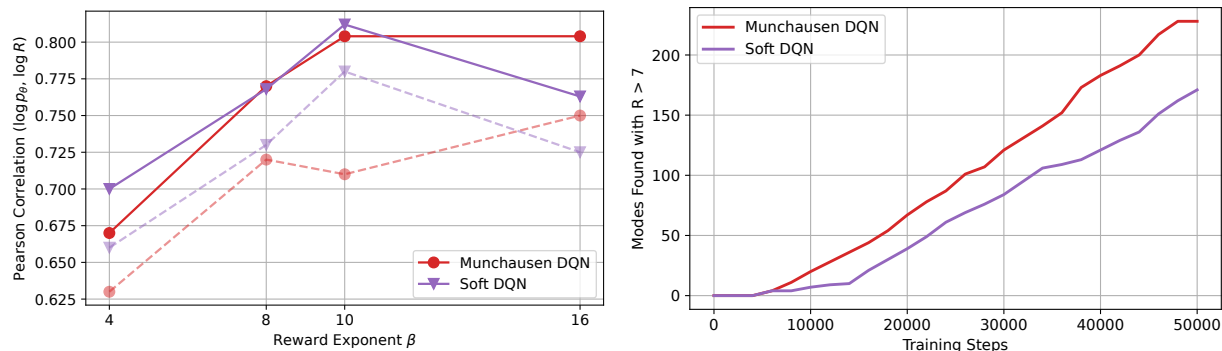


图 7：小分子生成结果。左图：每种方法在测试集上的 $\log \mathcal{R}$ 与 $\log \mathcal{P}_\theta$ 之间的皮尔逊相关性，以及变化的 $\beta \in \{4, 8, 10, 16\}$ 。实线表示在学习率选择下的最佳结果，虚线表示平均结果。右图：训练过程中发现的 Tanimoto 分离模式的数量，使用 $\mathcal{R} > 7.0$ ，对于 $\beta = 10$ 。

附录 E.3.3 位序列生成

最后，我们将 M-DQN 和 SoftDQN 在位序列生成任务上进行了比较。实验设置与第 4 节相同。图 8 展示了结果。与 M-DQN 相比，SoftDQN 具有相当或更差的奖励相关性，并且发现模式的速度稍慢。

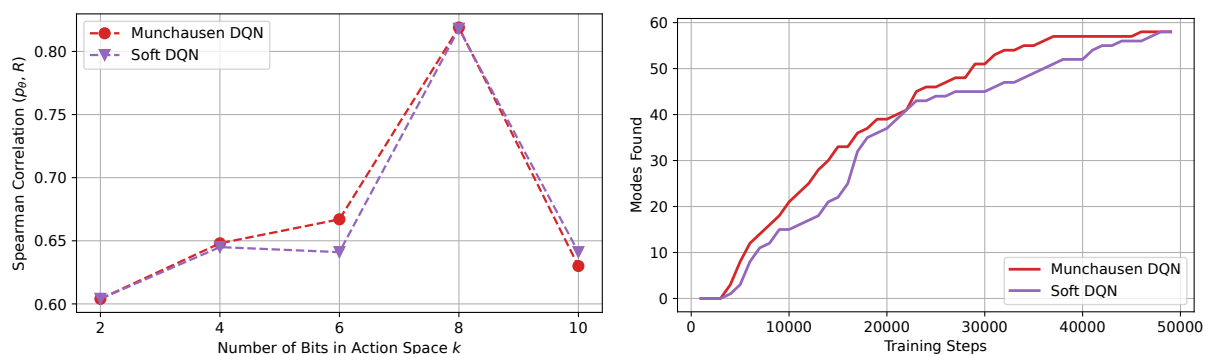


图 8：位序列生成结果。左图：每种方法在测试集上的 $\mathcal{R}$ 与 $\mathcal{P}_\theta$ 之间的斯皮尔曼相关性，以及变化的 $k \in \{2, 4, 6, 8, 10\}$ 。右图：训练过程中发现的模式数量，对于 $k = 8$ 。